

Influence Persistence Maximization in Temporal Social Networks

Xueqin Chang, Qing Liu, Baihua Zheng, Yunjun Gao, *Senior Member, IEEE*

Abstract—In this paper, we investigate a novel **Influence Persistence Maximization (INFPM)** problem in temporal social networks. Given a temporal graph, INFPM aims to identify a fixed seed node set S that maximizes the total duration of persistent influence across consecutive snapshots. After proving that INFPM is NP-hard, monotonic, and non-submodular, we develop two efficient solutions: (1) **RevG**, a reverse greedy algorithm that iteratively removes low-contribution nodes, and (2) **LRep**, a replacement-based method that progressively improves the quality of seed node set. To accelerate influence computation in **RevG** and **LRep**, we propose a new influence computation method integrating snapshot compression, probability-aware sampling, and a specialized influence estimator offering unbiased estimation. Additionally, we explore a practical variant of INFPM, termed **WIN-INFPM**, which relaxes the requirement of consecutive snapshots by introducing a flexible time window model. Extensive experiments on seven real-world networks demonstrate that (1) **RevG** and **LRep** effectively identify high-quality seed nodes, achieving up to 100% improvement in total influence persistence over the baselines; and (2) the proposed influence computation method improves the efficiency of **RevG** and **LRep** by up to 400%, while maintaining comparable influence persistence.

Index Terms—Persistence maximization, temporal social networks, theoretical analysis

I. INTRODUCTION

Influence Maximization (IM) [1] is an important task in social network analysis. Given a graph G and a budget k , IM aims to select a set of k seed nodes that maximizes the expected number of influenced nodes under a specified diffusion model. As a well-studied problem with significant practical implications, IM has broad applications in viral marketing [2], diffusion control [3], and social recommendation [4].

A large amount of work has focused on IM in static social networks [1], [5]–[9], where user relationships and diffusion patterns are assumed to remain constant over time. However, real-world social networks are *inherently temporal* [10], [11], with interactions occurring at specific timestamps and forming temporal sequences as the network evolves. For example, every minute, around 400 new users join Facebook and more than 510,000 comments are posted [12], each with precise timestamps. To capture such temporal characteristics, recent studies have explored IM in temporal social networks [13]. Existing works in this area can be mainly clustered into two categories: (1) selecting a seed node set to influence the maximum number of nodes at a specific timestamp [14]–[18], and (2) updating the seed node set at every timestamps to maximize its influence as the network evolves [19]–[23]. However, prior work suffers from two key limitations:

- 1) **Short-lived Influence**: The first category of work maximizes influence only at a single timestamp, creating transient influence spikes with small real-world impact [24].

Xueqin Chang, Qing Liu, and Yunjun Gao are with the Zhejiang University, Hangzhou 310027, China (e-mail: changxq@zju.edu.cn, qingliucs@zju.edu.cn, gaoyj@zju.edu.cn).

Baihua Zheng is with the Singapore Management University, Singapore 188065, (e-mail: bhzheng@smu.edu.sg).

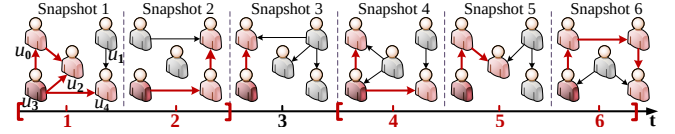


Fig. 1: A motivation example with $S = \{u_3\}$

In practical marketing campaigns, user adoption rarely occurs after a single exposure [25]. Effective adoption typically requires repeated exposures and sustained influence across multiple timestamps [26], [27].

- 2) **Costly and Impractical Updates**: The second category of work iteratively updates seed node set to match network snapshots, which is often infeasible in practice. Rapid network evolution, combined with the high costs of seed deployment (e.g., \$20K+ per campaign for top influencers [28]) and substantial computational cost for seed selection at each update, makes frequent updates prohibitively expensive.

The Influence Persistence Maximization Problem. To address the above limitations, we introduce a novel problem called **Influence Persistence Maximization (INFPM)** in temporal social networks. Specifically, given a temporal graph $\mathcal{G}$, represented as a sequence of snapshots, a budget k , an influence threshold α , and a persistence threshold θ , INFPM aims to identify a *fixed* seed node set $S \subset V$ ($|S| = k$) that maximizes *influence persistence*. We define the influence persistence as the total duration of all time intervals that satisfy the following conditions: (1) each interval spans at least θ consecutive timestamps; and (2) S influences at least an α -fraction of nodes in every snapshot within the interval. Here, α denotes the minimum influence fraction for meaningful impact [29], and θ represents the minimum persistence requirement, reflecting real-world needs for sustained exposure [30]. Figure 1 illustrates INFPM on a six-snapshot temporal network. With $k = 1$, $\alpha = 50\%$, and $\theta = 2$, a snapshot is valid when at least three nodes are influenced. As shown, when $S = \{u_3\}$, intervals $[1, 2]$ and $[4, 6]$ satisfy the INFPM conditions, yielding a maximum influence persistence of 5.

Applications. The INFPM problem has important practical implications, particularly in marketing. Studies indicate that repeated product exposure over consecutive days significantly increases customer familiarity and purchase likelihood [26]. On platforms such as Instagram and YouTube, companies seek to identify influential users who consistently reach and impact large audiences during specific campaign periods [31], [32]. INFPM addresses this by identifying users with maximum influence persistence over any given time spans. In practice, companies can leverage INFPM to analyze different temporal segments, such as holiday shopping seasons versus regular periods or quarterly intervals, to understand evolving influencer dynamics and optimize strategies. Moreover, when applied to temporal networks built from forecasted snapshots [33], INFPM enables proactive identification of users capable of

maintaining sustained influence in upcoming campaigns.

Challenges and Solutions. We prove that the INFPM problem is NP-hard and its objective function is monotone but non-submodular. Traditional greedy methods [1] for IM problem, which incrementally build the seed node set from an empty set, suffer from a *cold-start issue* in the INFPM setting. Specifically, when the seed node set is empty or very small, the influence persistence is often zero, since no qualified time intervals can be found. Consequently, the marginal gain in influence persistence (i.e., the increase in influence persistence from adding a new node to the current seed set) also becomes zero. This prevents traditional greedy methods from selecting effective seed nodes in the initial stage. To address this challenge, we propose two effective seed set selection algorithms: **ReverseGreedy (RevG)** and **LazyReplace (LRep)**. Unlike traditional greedy approaches that build seed node sets incrementally from scratch, both RevG and LRep are designed to maintain meaningful influence persistence throughout the selection process, thus avoiding the cold-start problem.

- **RevG** works in a top-down manner. It starts with a well-constructed candidate node set and iteratively removes the nodes that contribute the least to influence persistence until the set is reduced to size k . To generate a *high-quality candidate set*, we apply a multi-step filtering process: (1) Identify the size- k seed node set that maximizes influence for each snapshot. (2) Retain only those snapshots where (i) the seed set influences at least α -fraction of nodes, and (ii) there are at least θ such snapshots in succession. (3) Aggregate all the seed node sets of snapshots from the second step to form the initial candidate set. RevG then iteratively removes nodes from the candidate set, each time selecting the one whose removal minimizes the reduction in influence persistence, thereby preserving the overall influence persistence as much as possible.
- **LRep** implements a replacement-based optimization strategy. It starts with the top- k influential nodes, determined by their average influence spread across all snapshots, and iteratively refines this set by strategically replacing weaker seeds with stronger candidates to maximize influence persistence. To ensure computational efficiency, LRep uses a two-phase lazy evaluation strategy: (1) Rank candidate nodes based on their aggregated influence across all snapshots. (2) For each high-ranking candidate node n_c , first check whether replacing the *weakest seed* in the current seed set (i.e., the node contributing least to influence persistence) improves the overall influence persistence. If so, evaluate all nodes in the current seed set by considering their replacement with n_c and identify the one whose replacement yields the largest influence persistence. The identified seed is then swapped with n_c . The process continues until no further beneficial replacements are possible.

Both RevG and LRep require frequent computation of influence spread for the seed node set. Existing methods like Monte Carlo simulations [1] and Reverse Influence Sampling (RIS) [34] are computationally expensive in this setting. To overcome this bottleneck, we develop a more efficient influence spread computation method with three key innovations.

First, we introduce a snapshot compression technique that transforms each snapshot into a directed acyclic graph (DAG) by performing edge sampling and collapsing strongly connected components into single hypernodes. This reduces the graph size and simplifies its structure. Second, we propose a novel sampling technique for reverse reachable sets generation on the compressed DAG by considering different sampling probabilities for different hypernodes, unlike classical RIS which simply treats all nodes equally and samples them uniformly. Third, as the new sampling technique renders classical unbiased estimators ineffective, we design a specialized influence estimator tailored to the proposed sampling method to ensure unbiased estimation, with theoretical guarantees. For each snapshot G_t , the estimated influence spread lies within a factor of $(1 - \epsilon)$ and $(1 + \epsilon) \cdot (\prod_{j=1}^{\ell} \text{Scr}(G_t[C_j]))^{-1}$ of the true value, where $\epsilon \in (0, 1)$ and $\prod_{j=1}^{\ell} \text{Scr}(G_t[C_j])$ denotes the overall strongly connected reliability of G_t .

We also explore WIN-INFPM, a generalized variant of INFPM that relaxes the strict requirement of θ -consecutive-snapshots. In WIN-INFPM, a time interval is qualified if each time window within it contains at least θ snapshots in which the seed set influences at least an α -fraction of nodes. To solve WIN-INFPM efficiently, we adapt RevG and LRep by integrating a novel influence persistence computation algorithm PssW. This extension preserves the computational advantages of our original approaches while accommodating the more flexible temporal constraints of WIN-INFPM.

Contributions. Our contributions are summarized as follows.

- We formally define the INFPM problem for the first time, and prove its NP-hardness through rigorous analysis.
- We propose two effective seed selection algorithms, RevG and LRep, and develop an efficient influence computation method with provably unbiased estimation.
- We introduce WIN-INFPM, a practical extension that supports a flexible window-based influence persistence model.
- We conduct extensive experiments on seven real-world temporal network, demonstrating the effectiveness, efficiency, and scalability of our proposed algorithms.

Roadmap. Section II reviews the related work. Section III formally defines and analyzes the INFPM problem. Section IV presents RevG and LRep algorithms. Section V introduces influence computation algorithms. Section VI presents the WIN-INFPM variant and its solutions. Section VII reports experimental results, and Section VIII concludes the paper.

II. RELATED WORK

Influence Maximization in Static Social Networks. The Influence Maximization (IM) problem was first formulated as a discrete optimization problem by Kempe et al. [1], focusing on two fundamental propagation models: the Independent Cascade and Linear Threshold models. Although IM is NP-hard under both models, the monotonicity and submodularity of the objective function allow the use of greedy algorithms with a $(1 - 1/e)$ -approximation. Since then, substantial follow-up research has focused on developing efficient and scalable IM solutions [5]–[8], [35]–[37], with many approaches leveraging RIS techniques [34]. In parallel, researchers have proposed

a wide range of IM variants for specialized scenarios [38]–[45]. Additionally, [46]–[48] provide comprehensive surveys and experimental evaluation of IM.

Influence Maximization in Temporal Social Networks.

Recent research on influence maximization in temporal social networks, as reviewed in [13], falls into two main categories: (1) Snapshot-specific seed selection: These approaches select seed node sets to maximize influence at particular time instances [14]–[18]. For example, Zhao et al. [14], [15] proposed algorithms for seed selection at given query times while accounting for social activity decay. (2) Dynamic seed node set updating: Methods in this category focus on efficiently updating seed node sets to maximize influence as the network changes over time [19]–[23]. For instance, Ohsaka et al. [19] developed a dynamic scheme for seed node set updates with node/edge changes. It is worth mentioning that our work introduces *two fundamental differences* from existing IM research. (1) While prior studies focus on maximizing influence at specific timestamps, our objective is to maintain sufficient influence over multiple consecutive timestamps. (2) Whereas some existing approaches dynamically update the seed node set, we seek a single, fixed seed node set that maximizes influence persistence across the entire temporal network.

III. PRELIMINARIES

In this section, we formally define the Influence Persistence Maximization problem and analyze its fundamental properties.

A. Problem Formulation

Following prior work [20], [49], [50], we model the temporal social network as a sequence of snapshot graphs $\mathcal{G} = (V, E) = \{G_t\}_{t=1}^T = \{G_1, G_2, \dots, G_T\}$ ($t \in \mathbb{N}^+$). We assume that the set of nodes remains fixed across all snapshot graphs and the edges in each snapshot graph change over different timestamps. Each snapshot graph $G_t(V, E_t) \in \mathcal{G}$ is modeled as a directed graph, where V and E_t represent the sets of $|V|$ nodes and $|E_t|$ edges, respectively. The edges over all snapshot graphs comprise $E = \bigcup_{t=1}^T E_t$. For a directed edge $e_t = (u, v) \in E_t$, we refer to u as an in-neighbor of v and v as an out-neighbor of u . Correspondingly, we employ $N_t^{in}(v)$ (resp. $N_t^{out}(v)$) to denote the set of in-neighbors (resp. out-neighbors) of v in G_t . Each directed edge $e_t = (u, v) \in E_t$ is associated with an influence weight $w_t(u, v) \in [0, 1]$, quantifying the influence of node u on node v at timestamp t . Note that the influence weights $w_t(u, v)$ over different timestamp t may be different.

In each snapshot graph, the influence diffuses independently. In this paper, we consider two basic and widely adopted diffusion models, i.e., the independent cascade (IC) and linear threshold (LT) models [1]. Given a snapshot graph G_t and a set of seed nodes S , both models simulate influence spread through an iterative process. At step 0, the seed nodes in S are activated and the other nodes are inactive. The IC and LT models differ in the following steps. Specifically, in the IC model, when a node u is activated at step $i \geq 0$, it attempts to activate each inactive out-neighbor v at step $i + 1$ with probability $w_t(u, v)$, with only one attempt

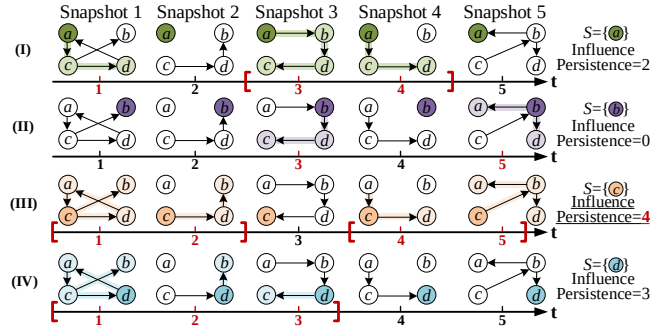


Fig. 2: A simple temporal graph

permitted. In the LT model, each node is associated with a uniformly sampled activation threshold $\rho_v^t \in [0, 1]$. At step $i > 0$, an inactive node v is activated iff the sum of influence weights from its activated in-neighbors is at least ρ_v^t , i.e., $\sum_{u \in N_t^{in}(v) \wedge u \text{ is activated}} w_t(u, v) \geq \rho_v^t$. Noticeably, the LT model requires that each node v satisfies $\sum_{u \in N_t^{in}(v)} w_t(u, v) \leq 1$. In both models, activated nodes remain active, and the process terminates when no further activations occur, yielding the set of eventually activated nodes, denoted as $I(G_t, S)$. Accordingly, the expected influence spread of S in snapshot G_t is $\sigma_t(S) = \mathbb{E}[|I(G_t, S)|]$.

Definition 1. (Maximal (α, θ, S) -persistent influence interval). Given a temporal graph $\mathcal{G} = \{G_t\}_{t=1}^T$, a seed node set $S \subset V$, and parameters $\alpha \in (0, 1]$ and $\theta \geq 1$ ($\theta \in \mathbb{N}^+$), a time interval $[t_s, t_e] \subseteq [1, T]$ is a maximal (α, θ, S) -persistent influence interval for $\mathcal{G}$ if the following conditions hold:

- 1) **α -valid:** $\forall t \in [t_s, t_e], \sigma_t(S) \geq \alpha \cdot |V|$.
- 2) **θ -persistent:** $|t_e - t_s + 1| \geq \theta$.
- 3) **Maximal:** $\nexists [t'_s, t'_e] \supset [t_s, t_e]$ satisfies conditions 1) and 2).

In other words, a maximal (α, θ, S) -persistent influence interval $[t_s, t_e]$ ensures that the expected influence spread of S meets the coverage threshold $\alpha \cdot |V|$ within the interval and that the interval spans at least θ consecutive timestamps. The interval is maximal in the sense that it cannot be extended further without violating these conditions. Note that multiple such intervals may exist for a given S . Consider the temporal graph $\mathcal{G}$ in Figure 2 (III), with $S = \{c\}$, $\alpha = 0.5$, and $\theta = 2$. The time intervals $[1, 2]$ and $[4, 5]$ are both valid maximal $(0.5, 2, \{c\})$ -persistent influence intervals since: (i) at each timestamp in these intervals, $\sigma_t(\{c\}) \geq 0.5 \cdot 4 = 2$, (ii) each interval has a length of 2, and (iii) neither intervals can be extended while satisfying the above two conditions.

Definition 2. (Influence persistence). Given a temporal graph $\mathcal{G} = \{G_t\}_{t=1}^T$, a seed node set $S \subset V$, and two parameters $\alpha \in (0, 1]$ and $\theta \geq 1$, let $\mathcal{T}(S, \mathcal{G}) = \{[t_{s1}, t_{e1}], \dots, [t_{sr}, t_{er}]\}$ be the set of all maximal (α, θ, S) -persistent influence intervals. The influence persistence of S in $\mathcal{G}$ w.r.t. α and θ , denoted by $F(\alpha, \theta, S)$, is:

$$F(\alpha, \theta, S) \triangleq \begin{cases} \sum_{i=1}^r (t_{ei} - t_{si} + 1), & \mathcal{T} \neq \emptyset \\ 0, & \mathcal{T} = \emptyset \end{cases} \quad (1)$$

That is, the influence persistence of S in a temporal graph $\mathcal{G}$ equals to the total length of all maximal (α, θ, S) -persistent influence intervals. We aggregate these lengths because maximizing *temporal coverage* is the key objective once α ensures sufficient influence at each timestamp. Continuing the example

from Figure 2 (III): we have $\mathcal{T}(\{c\}, \mathcal{G}) = \{[1, 2], [4, 5]\}$, and thus $F(0.5, 2, \{c\}) = 2 + 2 = 4$. Building upon the above definitions, we formally define the INFPM problem.

Problem 1. (INFPM). *Given a temporal graph $\mathcal{G} = \{G_t\}_{t=1}^T$, a budget k , and two parameters $\alpha \in (0, 1]$ and $\theta \geq 1$, the objective of the INFPM problem is to find a seed node set $S \subset V$ with $|S| = k$ that maximizes influence persistence of S . Formally,*

$$S = \arg \max_{S \subset V \wedge |S|=k} F(\alpha, \theta, S). \quad (2)$$

Example 1. *Consider the temporal graph $\mathcal{G}$ with five snapshots in Figure 2. Let $k = 1$, $\alpha = 0.5$, and $\theta = 2$. Among all seed sets of size 1, selecting $S = \{c\}$ yields the longest total duration of maximal $(0.5, 2, \{c\})$ -persistent influence intervals, namely $[1, 2]$ and $[4, 5]$, with $F(0.5, 2, \{c\}) = 4$. Hence, INFPM returns $S = \{c\}$.*

B. Problem Analyses

In this section, we first prove that INFPM is NP-hard, and then show that its influence persistence is monotonic but not submodular under both IC and LT models.

Theorem 1. *The INFPM problem is NP-hard.*

Proof. We prove this by reduction from the Decision version of the Set Cover (DSC) problem, which is known to be NP-complete. Given a ground set $U = \{u_1, u_2, \dots, u_n\}$ and a collection of subsets $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ where each $S_i \subseteq U$, the DSC problem asks whether there exist k subsets in $\mathcal{S}$ that together cover all elements in U . We construct an instance of INFPM as follows. Create a snapshot graph G_1 with two disjoint node sets: 1) $X = \{x_1, x_2, \dots, x_m\}$, where each x_i corresponds to a subset $S_i \in \mathcal{S}$; 2) $Y = \{y_1, y_2, \dots, y_n\}$, where each y_j corresponds to an element $u_j \in U$. For each pair (S_i, u_j) such that $u_j \in S_i$, add a directed edge from x_i to y_j with weight 1. The temporal graph $\mathcal{G}$ consists of a single snapshot G_1 . Next, we set the INFPM parameters: $\alpha = (k + n)/(m + n)$, $\theta = 1$, and k (same as in the DSC). Assume a deterministic propagation model where each seed node activates all its out-neighbors in a single timestamp. The influence persistence is 1 iff all nodes in Y are activated.

($\Rightarrow$) Suppose $S' \subset \mathcal{S}$ with $|S'| = k$ covers U . If we take all nodes x_j that represent the corresponding subsets $S_i \in S'$ as seed nodes, all nodes in Y can be activated.

($\Leftarrow$) Suppose $X' \subset X$ with $|X'| = k$ is the seed set that activate all nodes in Y . Then, the set $S' = \{S_i : x_i \in X'\}$ can cover U .

Therefore, INFPM can find k seed nodes if and only if the DSC has a solution, establishing NP-hardness. $\square$

Theorem 2. *Given a temporal graph $\mathcal{G} = \{G_t\}_{t=1}^T$ and two parameters α and θ , $F(\alpha, \theta, S)$ is monotone and non-submodular w.r.t. S under both IC and LT models.*

Proof. (1) **Monotonicity:** By the monotonicity property of influence spread [1], adding seed nodes can not decrease the expected influence spread under both IC and LT models. Thus, for any two seed sets $S_1 \subseteq S_2 \subseteq V$, we have $\sigma_t(S_1) \leq \sigma_t(S_2)$ for all $t \in [1, T]$. This implies $\mathcal{T}(S_1, \mathcal{G}) \subseteq \mathcal{T}(S_2, \mathcal{G})$. Consequently, any snapshot that contributes to $F(\alpha, \theta, S_1)$ will also contribute to $F(\alpha, \theta, S_2)$, ensuring that $F(\alpha, \theta, S_1) \leq F(\alpha, \theta, S_2)$. This establishes the monotonicity of $F(\alpha, \theta, S)$.

(2) **Non-submodularity:** For two sets $A, B \subseteq V$, if a function $f(\cdot)$ is submodular, it must satisfy $f(A) + f(B) \geq f(A \cup B) + f(A \cap B)$. Consider the temporal graph in Figure 2. Let $k = 1$, $\alpha = 50\%$, $\theta = 2$, $A = \{a\}$, and $B = \{b\}$. Then, $A \cap B = \emptyset$ and $A \cup B = \{a, b\}$. We have the influence persistences: $F(0.5, 2, \{a\}) = 2$; $F(0.5, 2, \{b\}) = 0$; $F(0.5, 2, \{a, b\}) = 3$; and $F(0.5, 2, \emptyset) = 0$. We have the following inequality: $F(0.5, 2, \{a\}) + F(0.5, 2, \{b\}) = 2 + 0 = 2 < F(0.5, 2, \{a \cup b\}) + F(0.5, 2, \emptyset) = 3 + 0 = 3$, indicating $f(A) + f(B) < f(A \cup B) + f(A \cap B)$. Therefore, $F(\alpha, \theta, S)$ is non-submodular w.r.t. S . $\square$

IV. INFPM ALGORITHMS

This section presents **ReverseGreedy** and **LazyReplace** algorithms to address the INFPM problem.

A. ReverseGreedy Algorithm

Given a snapshot graph G_t , let $\sigma_t(u|S) = \sigma_t(S \cup \{u\}) - \sigma_t(S)$ denote the marginal gain of adding u to the seed set S . Traditional greedy methods for IM [1] start from an empty set $S = \emptyset$ and iteratively add nodes with the highest marginal gain. However, this strategy faces significant limitations when applied to INFPM. Most notably, a *cold-start issue* arises: when the seed set is empty or contains only a few nodes, the influence persistence is likely zero. This is because isolated nodes often fail to generate sufficient influence to satisfy the threshold α across snapshots, preventing the formation of qualified persistent intervals. Consequently, the marginal gain for any node becomes zero, hindering effective early-stage selections. Additionally, INFPM requires evaluating marginal gains across multiple snapshots instead of a single one, which adds complexity. Moreover, the INFPM objective is non-submodular as demonstrated in Theorem 2, undermining the effectiveness and $(1-1/e)$ approximation of greedy algorithms.

To overcome these limitations, we propose **ReverseGreedy (RevG)**, a fundamentally different strategy. Instead of building up from an empty set, **RevG** starts with a candidate set C (where $|C| \geq k$) that already achieves substantial influence spread. It then iteratively removes the node contributing least to the overall influence persistence, continuing until exactly k nodes remain. The final set is returned as the solution. However, applying **RevG** effectively requires tackling two main challenges. (1) **Candidate Set Construction:** A larger candidate set offers greater potential to include high-quality seed nodes but requires more pruning iterations. A smaller set reduces computational cost but risks missing influential nodes. Therefore, constructing a candidate set that balances coverage and size is critical. (2) **Snapshot Evaluation Cost:** Temporal graphs typically comprise many snapshots. Each node removal decision requires verifying that the influence threshold α remains satisfied across *all snapshots*. Repeating this verification for every iteration and every snapshot introduces substantial computational overhead. Next, we introduce how to address these two challenges in detail.

Constructing the initial candidate set. To address the first challenge, we design the **Candidate Generation** algorithm, which intelligently constructs a candidate set C^* , consisting

Algorithm 1 Candidate Generation (CandGen)

Input: Temporal graph $\mathcal{G} = \{G_t\}_{t=1}^T$, budget k , threshold α , persistence θ

Output: Candidate set C^* , valid timestamp set T^*

```

1: Initialize  $C^* \leftarrow \emptyset$ ;  $T_\alpha^U \leftarrow \emptyset$ ,  $T_\theta^U \leftarrow \emptyset$ ;
2: for each timestamp  $t = 1$  to  $T$  do
3:   Select the optimal  $k$ -seed node set  $S_t$  from  $G_t$ ;
4:   if  $\sigma_t(S_t) \geq \alpha \cdot |V|$  then  $T_\alpha^U \leftarrow T_\alpha^U \cup \{t\}$ ; //Theorem 3
5: if  $|T_\alpha^U| \geq \theta$  then
6:    $T_\theta^U \leftarrow \text{GetPersistence}(T_\alpha^U, \theta, T_\theta^U)$ ;
7:   for each timestamp  $t \in T_\theta^U$  do  $C^* \leftarrow C^* \cup S_t$ ; //Theorem 4
8: else  $C^* \leftarrow \emptyset$ ;
9: if  $|C^*| > \alpha \cdot |V|$  then
10:  Let  $C^*$  be the top  $\alpha \cdot |V|$  most frequent nodes in  $\{S_t | t \in T_\theta^U\}$ ;
11:  $T^* \leftarrow T_\theta^U$ ;
12: Return  $C^*$ ,  $T^*$ .

```

Procedure GetPersistence (T_X^α , θ , T_X^θ):

```

1: Sort all timestamp  $t$  in  $T_X^\alpha$  in ascending order;
2:  $F(\alpha, \theta, X) \leftarrow 0$ ,  $\mathcal{T}(X, \mathcal{G}) \leftarrow \emptyset$ ,  $s \leftarrow 0$ ;
3: for  $i \leftarrow 1$  to  $|T_X^\alpha|$  do
4:   if  $i = |T_X^\alpha|$  or  $T_X^\alpha[i] \neq T_X^\alpha[i-1] + 1$  then
5:     if  $i - s \geq \theta$  then
6:        $\mathcal{T}(X, \mathcal{G}) \leftarrow \mathcal{T}(X, \mathcal{G}) \cup \{[T_X^\alpha[s], T_X^\alpha[i-1]]\}$ ;
7:        $T_X^\theta \leftarrow T_X^\theta \cup \{t \mid T_X^\alpha[s] \leq t \leq T_X^\alpha[i-1]\}$ ;
8:        $s \leftarrow i$ ;
9:  $F(\alpha, \theta, X) \leftarrow \sum_{[t_h, t_t] \in \mathcal{T}(X, \mathcal{G})} (t_t - t_h + 1)$ ;
10: Return  $F(\alpha, \theta, X)$ ,  $T_X^\theta$ .

```

of nodes likely to be selected as seeds. The key intuition is that nodes appearing in optimal k -seed sets that maximize influence in individual snapshots are more likely to be part of a globally effective seed set. However, directly taking the union of these seed sets may yield a large candidate set. To control its size, we introduce two pruning strategies based on the following theorems.¹

Theorem 3. Given a temporal graph $\mathcal{G}$ and influence threshold α , if a snapshot $G_t = (V, E_t) \in \mathcal{G}$ satisfies $\max_{S' \subseteq V, |S'|=k} \sigma_t(S') < \alpha \cdot |V|$, no seed set of size k can meet the influence threshold in G_t , and G_t can be safely pruned.

Theorem 3 indicates that if the upper bound of influence spread in a snapshot G_t (i.e., $\max_{S' \subseteq V, |S'|=k} \sigma_t(S')$) is less than the threshold $\alpha \cdot |V|$, then G_t can be safely pruned. Based on Theorem 3, we define the upper-bound set of α -valid timestamps as $T_\alpha^U = \{t \in [T] \mid \max_{S \subseteq V, |S|=k} \sigma_t(S) \geq \alpha \cdot |V|\}$.

Theorem 4. Given a temporal graph $\mathcal{G}$ with parameters α , θ , and the upper-bound set α -valid timestamps T_α^U , let T_θ^U denote the upper-bound set θ -valid timestamps, $T_\theta^U = \cup \{I \subseteq T_\alpha^U \mid I \text{ is a maximal persistent interval with } |I| \geq \theta\}$. For any $t \in T_\alpha^U \setminus T_\theta^U$, no seed set of size k can satisfy the persistence requirement at G_t , and thus G_t can be safely pruned.

Theorem 4 proves that G_t can be safely pruned if timestamp t does not belong to a maximal persistent interval, even under the upper-bound influence spread. This is because such a snapshot cannot contribute to any valid persistence interval. Notably, the pruning operations defined by Theorems 3 and 4 are safe: all pruned snapshots are guaranteed not to contribute to influence persistence and can be excluded from influence spread evaluations for every candidate seed set.

Based on these two theorems, Algorithm 1 presents the Candidate Generation (CandGen) algorithm. It first initial-

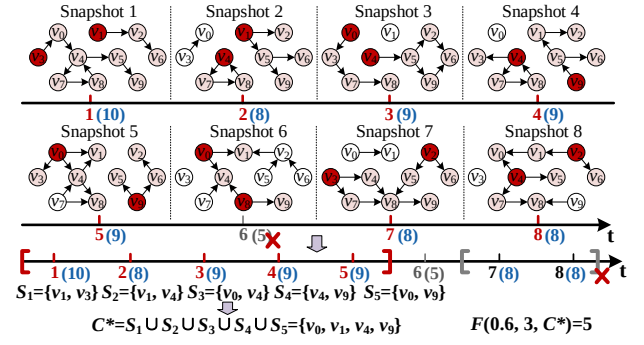


Fig. 3: An example of CandGen Algorithm

izes empty sets C^* , T_α^U , and T_θ^U (Line 1). For each snapshot G_t , it computes the optimal k -seed set S_t using the greedy algorithm [1] with $(1 - 1/e)$ -approximation (Line 3). If S_t 's upper-bound influence spread satisfies threshold α , timestamp t is added to T_α^U (Line 4). Once $|T_\alpha^U| \geq \theta$, it invokes GetPersistence to obtain T_θ^U (Line 6) and constructs C^* as the union of all S_t where $t \in T_\theta^U$ (Line 7); otherwise, it returns an empty set (Line 8). If $|C^*| > \alpha \cdot |V|$, then C^* 's influence automatically meets threshold α regardless of which node is removed. Therefore, only the top $\alpha \cdot |V|$ most frequent nodes across all S_t are retained (Lines 9-10). Finally, the algorithm sets $T^* = T_\theta^U$ (Line 11) and returns C^* and T^* (Line 12).

The GetPersistence procedure computes the influence persistence $F(\alpha, \theta, X)$ and the corresponding θ -valid timestamp set T_X^θ for a given set of α -valid timestamps T_X^α . It first sorts T_X^α in ascending order and initializes variables (Lines 1-2), then scans for consecutive intervals (Lines 3-8). When a discontinuity is detected (Line 4), if the interval length exceeds θ (Line 5), the interval is added to $\mathcal{T}(X, \mathcal{G})$ and its timestamps are inserted into T_X^θ (Lines 6-7). The interval start position is then reset (Line 8). Finally, the procedure returns the influence persistence and the θ -valid timestamp set (Lines 9-10).

Example 2. Consider the temporal graph $\mathcal{G}$ with eight snapshots in Figure 3, where $\alpha = 0.6$, $\theta = 3$, and $k = 2$. CandGen first selects the optimal seed node set S_i ($|S_i| = 2$) that maximizes influence spread for each snapshot $G_i \in \mathcal{G}$. Since $\sigma(S_6) = 5 < 0.6 \times 10 = 6$, snapshot G_6 is pruned. It then computes persistent intervals over $T_\alpha = \{1, 2, 3, 4, 5, 7, 8\}$. Since interval $[7, 8]$ has length shorter than $\theta = 3$, timestamps 7 and 8 are discarded, yielding $T_\theta = \{1, 2, 3, 4, 5\}$. The union of all selected seed node sets over T_θ forms the initial candidate set $C^* = \{v_0, v_1, v_4, v_9\}$ with $F(0.6, 3, C^*) = 5$.

Greedy node removal with snapshot pruning. After constructing the initial candidate set, we next iteratively remove nodes to obtain the final k seed nodes, each time selecting the one whose removal minimizes the reduction in influence persistence. However, each removal decision requires verifying that the influence threshold α remains satisfied across all snapshots. Since temporal graphs typically comprise many snapshots, repeating this verification for every iteration introduces substantial computational overhead. To address this challenge, we introduce a theorem that exploits the candidate set at each iteration to identify and exclude snapshots unlikely to contribute to influence persistence in subsequent steps.

Theorem 5. Given a snapshot graph $G_t \in \mathcal{G}$, a node set

¹Due to the space limitation, all omitted proofs are moved to the Appendix.

Algorithm 2 ReverseGreedy (RevG)

Input: Temporal graph $\mathcal{G} = \{G_t\}_{t=1}^T$, candidate set C^* , valid timestamp set T^* , budget k , threshold α , persistence θ
Output: Seed node set S

```

1: Initialize  $S \leftarrow \emptyset$ ,  $C \leftarrow C^*$ ,  $T_{\min} \leftarrow T^*$ ;
2: while  $|C| > k$  do
3:   for each node  $v \in C$  do
4:      $C' \leftarrow C \setminus \{v\}$ ,  $T_{C'}^\alpha \leftarrow \emptyset$ ,  $T_{C'}^\theta \leftarrow \emptyset$ ;
5:     for each timestamp  $t \in T_{\min}$  do //Theorem 5
6:       if  $\sigma_t(C') \geq \alpha \cdot |V|$  then  $T_{C'}^\alpha \leftarrow T_{C'}^\alpha \cup \{t\}$ ;
7:       if  $T_{C'}^\alpha \neq \emptyset$  then  $F, T_{C'}^\theta \leftarrow \text{GetPersistence}(T_{C'}^\alpha, \theta, T_{C'}^\theta)$ ;
8:       else  $F(\alpha, \theta, C') \leftarrow 0$ ;
9:        $\Delta(v|C) \leftarrow F(\alpha, \theta, C) - F(\alpha, \theta, C')$ ;
10:      if  $\Delta(v|C) = 0$  then break;
11:    $v^* \leftarrow \arg \min_{v \in C} \Delta(v|C)$ ,  $C \leftarrow C \setminus \{v^*\}$ ;
12:   Update  $F(\alpha, \theta, C) \leftarrow F(\alpha, \theta, C \setminus \{v^*\})$ ,  $T_{\min} \leftarrow T_{C \setminus \{v^*\}}^\theta$ ;
13:  $S \leftarrow C$ ;
14: Return  $S$ .
```

$A \subset V$, parameters α and θ , let $\mathcal{T}(A, \mathcal{G})$ denote the set of all maximal (α, θ, A) -persistent influence intervals. If timestamp $t \notin \bigcup_{I \in \mathcal{T}(A, \mathcal{G})} I$, then $\forall A' \subset A$, we have $t \notin \bigcup_{I' \in \mathcal{T}(A', \mathcal{G})} I'$.

Theorem 5 indicates that if a timestamp t is not covered by any maximal (α, θ, C) -persistent influence interval of the current candidate set C , it will not be covered by intervals of any subset $C' \subset C$. Therefore, t cannot contribute to the persistence objective during node removal and the corresponding snapshot G_t can be safely pruned from further evaluation.

Based on above considerations, we propose the RevG algorithm, with its pseudo-code presented in Algorithm 2. It initializes $S = \emptyset$, $C = C^*$, and $T_{\min} = T^*$, where $T_{\min}$ records timestamps contributing to persistence (Line 1). RevG iteratively removes nodes from C until $|C| = k$ (Lines 2-12). For each node $v \in C$, it checks whether influence spread after removing v still meets threshold α across all snapshots (Lines 5-6). If $T_{C'}^\alpha$ is non-empty, GetPersistence computes $F(\alpha, \theta, C')$ and updates $T_{C'}^\theta$ (Line 7); otherwise, persistence is set to zero (Line 8). The marginal loss $\Delta(v|C)$ is then calculated for each node (Line 9). Once $\Delta(v|C) = 0$ is found, the for loop (Lines 3-9) terminates early. Subsequently, the node v^* with minimum marginal loss is removed (Line 11), and $F(\alpha, \theta, C)$ and $T_{\min}$ are updated accordingly (Line 12). Finally, the remaining k nodes in C form the seed set S , which is returned as the solution (Lines 13-14).

Example 3. Reconsider the temporal graph $\mathcal{G}$ in Figure 3 with candidate set $C^* = \{v_0, v_1, v_4, v_9\}$ and $F(0.6, 3, C^*) = 5$. As shown in Figure 4, RevG iteratively removes nodes from C^* : removing v_4 incurs zero marginal loss, achieving optimality without examining v_9 , while removing v_1 yields the valid interval $[3, 5]$ with $F(0.6, 3, C^*) = 3$, resulting in the final seed set $S = \{v_0, v_9\}$.

Time complexity. The time complexity of constructing candidate set C^* is $O(T \cdot \varphi)$, where φ is the cost of computing influence spread per snapshot (Section V). In each of the $|C^*| - k$ iterations, RevG computes persistence loss for all nodes in C^* , requiring $O(T \cdot \varphi)$ time per iteration. Thus, the overall time complexity is $O(|C^*| \cdot (|C^*| - k) \cdot T \cdot \varphi)$.

B. LazyReplace Algorithm

In this subsection, we introduce LazyReplace (LRep), a new algorithm that identifies the optimal seed set through

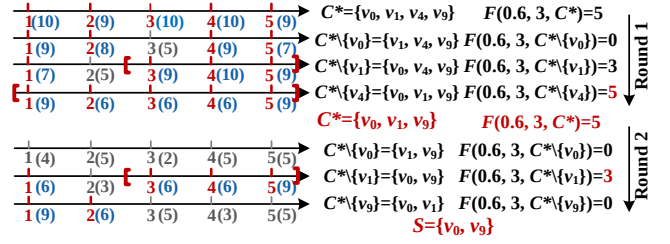


Fig. 4: An example of RevG Algorithm

a replacement-based approach to maximize influence persistence. The core idea of LRep is to initialize a seed node set with nodes whose individual influence spread is consistently high across all snapshots, then iteratively replace seed nodes with candidates that improve the influence persistence of the seed set. However, exhaustively evaluating all possible replacements incurs significant computational overhead. To address this, LRep adopts a lazy evaluation strategy with early termination: it prioritizes replacing the weakest node (i.e., the one contributing least to influence persistence) in the current seed set with each candidate. If this replacement fails to improve influence persistence, all subsequent replacements involving that candidate are skipped, thereby avoiding redundant calculations across all snapshots. The algorithm leverages the intuition that nodes with consistently strong influence are more likely to contribute to the optimal solution, while the lazy replacement strategy ensures efficiency by avoiding unnecessary evaluations.

LRep needs to address three key issues: (1) How to efficiently construct an initial seed node set for replacement? (2) How to prioritize candidate nodes to accelerate convergence? and (3) How to identify the weakest contributor in the current seed set for lazy replacement? First, we adopt a heuristic that computes the influence spread of each node $v \in V$ across all snapshots $G_t \in \mathcal{G}$ and selects the top- k nodes with the highest average influence. This approach is based on the intuition that nodes with greater average influence are more likely to contribute significantly to overall persistence. Second, we rank the remaining candidate nodes in descending order of their average single-node influence and perform replacements following this order, prioritizing nodes with higher potential influence spread. Third, following the principle used in RevG, we define the weakest contributor as the node whose removal results in the smallest decrease in influence persistence.

Algorithm 3 presents the pseudo-code of LRep, which identifies the optimal seed set for maximizing influence persistence. LRep first computes each node's average influence $\bar{\sigma}(v)$ across all snapshots (Lines 1-2), sorts nodes by descending average influence and selects the top- k nodes to form the initial seed set S (Line 3), and calculates the initial influence persistence $F(\alpha, \theta, S)$ (Line 4). It then iteratively refines S through lazy evaluation with early pruning (Lines 5-18). For each candidate node $v \in V \setminus S$, LRep identifies the weakest contributor s^* in S by invoking Lines 3-11 of Algorithm 2 (Line 6), then tests whether replacing s^* with v improves persistence. If no improvement is found (Line 9), all further evaluations involving v are pruned (Line 10). Otherwise, it exhaustively evaluates all replacements within S , tracking the best seed set

Algorithm 3 LazyReplace (LRep)

Input: Temporal graph $\mathcal{G} = \{G_t\}_{t=1}^T$, budget k , threshold α , persistence θ

Output: Seed node set S

```

1: for each node  $v \in V$  do
2:   Compute average influence  $\bar{\sigma}(v) = \frac{1}{T} \sum_{t=1}^T \sigma_t(v)$ ;
3:  $S \leftarrow$  top- $k$  nodes from  $V$  sorted by  $\bar{\sigma}(v)$  in descending order;
4:  $F(\alpha, \theta, S) \leftarrow \text{ComputePersistence}(\mathcal{G}, S, \alpha, \theta)$ ;
5: for each candidate node  $v \in V \setminus S$  do
6:    $s^* \leftarrow \text{FindWeakestNode}(S)$ ;
7:    $S' \leftarrow S \setminus \{s^*\} \cup \{v\}$ ;
8:    $F'(\alpha, \theta, S') \leftarrow \text{ComputePersistence}(\mathcal{G}, S', \alpha, \theta)$ ;
9:   if  $F'(\alpha, \theta, S') \leq F(\alpha, \theta, S)$  then
10:    Continue; // Prune exhaustive search
11:    $\hat{S} \leftarrow S$ ,  $\hat{F}(\alpha, \theta, \hat{S}) \leftarrow F(\alpha, \theta, S)$ ;
12:   for each node  $s \in S$  do
13:      $\tilde{S} \leftarrow S \setminus \{s\} \cup \{v\}$ ;
14:      $\tilde{F}(\alpha, \theta, \tilde{S}) \leftarrow \text{ComputePersistence}(\mathcal{G}, \tilde{S}, \alpha, \theta)$ ;
15:     if  $\tilde{F}(\alpha, \theta, \tilde{S}) > \hat{F}(\alpha, \theta, \hat{S})$  then
16:        $\hat{S} \leftarrow \tilde{S}$ ,  $\hat{F}(\alpha, \theta, \hat{S}) \leftarrow \tilde{F}(\alpha, \theta, \tilde{S})$ ;
17:   if  $\hat{F}(\alpha, \theta, \hat{S}) > F(\alpha, \theta, S)$  then
18:      $S \leftarrow \hat{S}$ ,  $F(\alpha, \theta, S) \leftarrow \hat{F}(\alpha, \theta, \hat{S})$ ;
19: Return  $S$ .
Function ComputePersistence( $\mathcal{G}, S, \alpha, \theta$ ):
20:  $T_S^\alpha \leftarrow \emptyset$ ,  $T_S^\theta \leftarrow \emptyset$ ;
21: for  $t = 1$  to  $T$  do
22:   if  $\sigma_t(S) \geq \alpha \cdot |V|$  then  $T_S^\alpha \leftarrow T_S^\alpha \cup \{t\}$ ;
23: if  $T_S^\alpha \neq \emptyset$  then Return GetPersistence( $T_S^\alpha, \theta, T_S^\theta$ );
24: else Return 0.

```

$\hat{S}$ and corresponding persistence $\hat{F}(\alpha, \theta, \hat{S})$ (Lines 12-16). If improvement occurs, S is updated accordingly (Lines 17-18). The auxiliary function `ComputePersistence` (Lines 20-24) calculates influence persistence.

Example 4. Consider the temporal graph $\mathcal{G}$ with five snapshots in Figure 5, where $\alpha = 0.6$, $\theta = 3$, and $k = 2$. *LRep* first computes each node's average influence across snapshots and selects the top-2 nodes as the initial seed set $S = \{v_0, v_2\}$ with $\bar{\sigma}(v_0) = 2.4$ and $\bar{\sigma}(v_2) = 2.2$. It then examines each candidate in $C = V \setminus S = \{v_1, v_3, v_4\}$, sorted by descending average influence. For each candidate, *LRep* tests replacements with seeds in S . For instance, replacing v_2 with v_1 improves persistence from 0 to 3. Subsequently, replacing v_0 with v_3 further increases persistence from 3 to 5. Thus, *LRep* returns $S = \{v_3, v_1\}$ with persistence 5 as the final solution.

Time complexity. Estimating average influence over T snapshots requires $O(|V| \cdot T \cdot \varphi)$ time. For each of the $|V| - k$ candidates, *LRep* performs up to $k + 1$ evaluations, each costing $O(T + \varphi \cdot k)$. Thus, the overall complexity of *LRep* is $O(|V| \cdot T \cdot \varphi + |V| \cdot k(T + \varphi \cdot k))$.

V. INFLUENCE COMPUTATION

Both *RevG* and *LRep* require numerous computations of influence spread across multiple snapshots to evaluate whether a given node set satisfies the influence threshold. However, computing the exact influence spread $\sigma_t(S)$ for any node set $S \subseteq V$ and snapshot $G_t \in \mathcal{G}$ is $\#P$ -hard under both the IC and LT models. This is because each snapshot G_t functions as a static graph, and the $\#P$ -hardness of exact influence spread computation for such static graphs has been rigorously established in prior works [1], [51]. Therefore, we employ approximation methods to estimate the value of $\sigma_t(S)$. In the

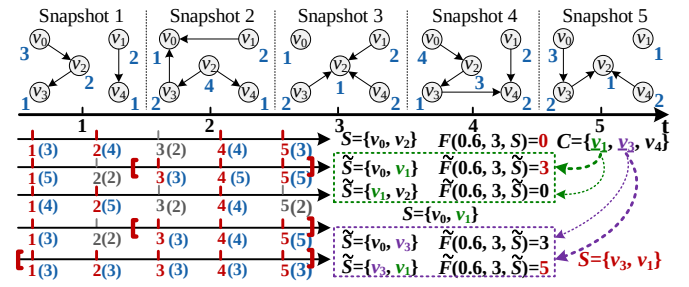


Fig. 5: An example of LRep Algorithm

following, we focus on estimating influence spread on a single snapshot, noting that this approach can be applied to the entire temporal graph.

Recent studies have employed sampling-based approaches for influence spread estimation, ranging from Monte-Carlo (MC) simulations [1] to Reverse Influence Sampling (RIS) [34]. Given that RIS significantly improves the efficiency of estimating influence spread compared to MC simulations, we adopt RIS to estimate $\sigma_t(S)$, consistent with state-of-the-art methods in influence estimation [6], [37]. Each reverse reachable (RR) set R sampled by RIS represents a subset of nodes that can reach a randomly selected node through a probabilistically sampled graph. Given a snapshot G_t , a seed set S , and a random RR set R generated on G_t , we define a random variable $\Upsilon_t(S, R)$ such that $\Upsilon_t(S, R) = 1$ if $S \cap R \neq \emptyset$ and $\Upsilon_t(S, R) = 0$ otherwise. Borgs et al. [34] proved that $\sigma_t(S)$ equals $\mathbb{E}[\Upsilon_t(S, R)] \cdot |V|$ under the triggering model. Given a set $\mathcal{R}_t = \{R_1, R_2, \dots\}$ of RR sets sampled from snapshot G_t , $\mathbb{E}[\Upsilon_t(S, R)] \cdot |V|$ can be unbiasedly estimated using the empirical mean $\sum_{R \in \mathcal{R}_t} \Upsilon_t(S, R) / |\mathcal{R}_t| \cdot |V|$ based on concentration bounds. Therefore, when $\mathcal{R}_t$ is fixed, we define $f_{\mathcal{R}_t}(S) = \frac{\sum_{R \in \mathcal{R}_t} \Upsilon_t(S, R)}{|\mathcal{R}_t|} \cdot |V|$ as an unbiased estimator of $\sigma_t(S)$ under the RIS framework.

While RIS enables efficient influence estimation for a given seed set S on a single snapshot, it faces significant challenges when applied to temporal graphs. In temporal graphs, each snapshot exhibits distinct structural characteristics that necessitate independent influence estimations for the seed set S at every timestamp. This computational overhead becomes prohibitive in real-world scenarios where rapid network dynamics generate extensive sequences of snapshots. To this end, we propose a novel snapshot compression technique that significantly reduces the size of each snapshot, thereby shortening sampling paths and minimizing the number of nodes visited per RR set under RIS. Our approach applies edge sampling followed by strongly connected component (SCC) decomposition, which aggregates nodes into SCCs to simplify the graph structure. Each snapshot G_t is transformed into a compressed directed acyclic graph (SCC-DAG), denoted as D_t . Based on this structure, we extend the classical RIS method to operate on SCC-DAGs by sampling SCC-based RR sets, referred to as SCC-RR sets. We further develop an unbiased influence estimator based on SCC-RR sets, which provides theoretical guarantees. This framework involves addressing three key challenges: (1) designing an effective node partition strategy to transform snapshots into compact SCC-DAGs; (2) developing efficient algorithms for sampling random SCC-RR sets from

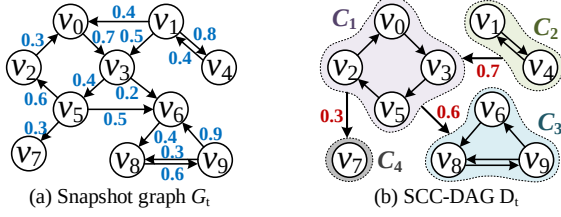


Fig. 6: Illustration of the snapshot compression method

the compressed structure; and (3) establishing an unbiased estimator for $\sigma_t(S)$ with theoretical guarantees.

(1) Constructing compressed snapshots SCC-DAGs. We begin by introducing a method for merging nodes into SCCs. Since computing strong connectivity reliability is $\#P$ -hard [52], [53] and approximate estimation via sampling remains computationally expensive, we adopt a heuristic approach. Given a snapshot $G_t \in \mathcal{G}$, we sample r random graphs by retaining all nodes while independently removing each edge $e_t = (u, v)$ with probability $1 - w_t(u, v)$. We then apply SCC decomposition to each sampled graph and select the partition with the fewest components (i.e., the coarsest). The intuition is that fewer SCCs imply larger strongly connected regions, which better preserve the potential for influence propagation and approximate an upper-bound structure of strong connectivity. We formalize this intuition in Definition 3.

Definition 3. (SCC partition P_t). Given a snapshot $G_t \in \mathcal{G}$, let $G_t^1 = (V, E_t^1), \dots, G_t^r = (V, E_t^r)$ be r random graphs independently sampled from G_t . Let $P_t^i = \{C_1, \dots, C_\ell\}$ denote the SCC decomposition of G_t^i , then the SCC partition of the node set V is chosen such that $|P_t^*| = \min_{i \in [r]} |P_t^i|$.

Given the SCC partition $P_t \in \mathcal{P}$ for each snapshot G_t , we construct its corresponding SCC-DAG D_t [54], as outlined in Definition 4.

Definition 4. (SCC-DAG). Given a snapshot G_t and its corresponding SCC partition $P_t = \{C_j\}_{j \in [\ell]}$, the compressed snapshot (SCC-DAG) derived from G_t is defined as $D_t(\Lambda_t, \Phi_t, p_t)$, where:

$$\begin{aligned} \Lambda_t &= \{C_j \mid j \in [\ell]\}, \\ \Phi_t &= \{\tilde{e}_t = (C_i, C_j) \mid C_i \neq C_j, \exists (u, v) \in E_t, u \in C_i, v \in C_j\}, \\ \forall (C_i, C_j) \in \Phi_t, p_t(C_i, C_j) &= 1 - \prod_{u \in C_i, v \in C_j} (1 - w_t(u, v)). \end{aligned}$$

Here, Λ_t represents the node set of hypernodes (SCCs), Φ_t denotes the edge set between distinct components, and $p_t(C_i, C_j)$ is the aggregated influence weight from component C_i to C_j .

Example 5. Figure 6 illustrates a snapshot graph $G_t = (V, E_t)$ and its corresponding SCC-DAG $D_t(\Lambda_t, \Phi_t, p_t)$. In this example, assuming the SCC partition is $P_t = \{C_1, C_2, C_3, C_4\}$, then we have $\Lambda_t = \{C_1, C_2, C_3, C_4\}$ where $C_1 = \{v_0, v_2, v_3, v_5\}$, $C_2 = \{v_1, v_3\}$, $C_3 = \{v_6, v_8, v_9\}$, and $C_4 = \{v_7\}$. The inter-component edges are represented as $\Phi_t = \{(C_1, C_4), (C_1, C_3), (C_2, C_1)\}$. The influence weight on an edge (C_i, C_j) in D_t aggregates the influence weights of all edges connecting nodes from components C_i to C_j in the original graph G_t . For instance, consider the two edges connecting C_1 to C_3 with individual influence weights of 0.2

Algorithm 4 Influence Estimation Framework

Input: A snapshot $G_t \in \mathcal{G}$, a seed set S , and a sample size $\mathcal{I}_t$
Output: Estimated influence $f_{\tilde{\mathcal{R}}_t}(S)$
1: $P_t \leftarrow$ Compute SCC partition of G_t by Definition 3;
2: $D_t \leftarrow$ Build SCC-DAG $D_t(\Lambda_t, \Phi_t, p_t)$ from G_t and P_t by Definition 4;
3: $\tilde{\mathcal{R}}_t \leftarrow$ Sample $\mathcal{I}_t$ random SCC-RR sets from D_t ;
4: $f_{\tilde{\mathcal{R}}_t}(S) \leftarrow$ Estimate influence spread of S using $\tilde{\mathcal{R}}_t$ by Eq.(4);
5: **Return** $f_{\tilde{\mathcal{R}}_t}(S)$.

and 0.5, respectively. The aggregated influence weight of edge (C_1, C_3) is calculated as $1 - (1 - 0.5)(1 - 0.2) = 0.6$.

(2) Sampling random SCC-RR sets on SCC-DAG. Building upon the above definitions, we extend the classical RIS framework to sample SCC-based reverse reachable (SCC-RR) sets on SCC-DAG. Given an SCC-DAG $D_t(\Lambda_t, \Phi_t, p_t)$, each sampled SCC-RR set $\tilde{R} \subseteq \Lambda_t$ is generated as follows:

- 1) Generate a random graph D'_t from D_t by retaining all hypernodes (i.e., SCCs) while independently removing each hyperedge $\tilde{e}_t = (C_i, C_j)$ with probability $1 - p_t(C_i, C_j)$;
- 2) Select a hypernode C_i from D'_t with probability $|C_i|/|V|$;
- 3) Let $\tilde{R}$ be the set of all hypernodes $C_j \in \Lambda_t$ such that there exists a directed path to C_i in D'_t .

Since $\Lambda_t = \{C_1, C_2, \dots, C_\ell\}$ forms a partition of the node set V , selecting a hypernode C_i with probability $|C_i|/|V|$ is mathematically equivalent to uniform node sampling in the original graph G_t . This enables us to build SCC-RR sets on the SCC-DAG while preserving the uniform sampling assumption of the traditional RIS, ensuring that the influence estimator remains unbiased.

(3) Estimating $\sigma_t(S)$ via sampled SCC-RR sets. Given a set of SCC-RR set $\tilde{\mathcal{R}}_t = \{\tilde{R}_1, \tilde{R}_2, \dots\}$ sampled from D_t , we present an unbiased method for estimating the influence spread of a seed node set S . For a snapshot graph G_t , a seed node set S , and a random SCC-RR set $\tilde{R}$, we define a random variable $\Omega_t(S, \tilde{R})$ as follows:

$$\Omega_t(S, \tilde{R}) = \begin{cases} 1 & \text{if } \exists s \in S, \exists C_i \in \tilde{R}, \text{ s.t. } s \in C_i, \\ 0 & \text{otherwise.} \end{cases} \quad (3)$$

This variable captures whether the seed set S intersects with the SCC-RR set $\tilde{R}$. The formulation is valid because strong connectivity within each hypernode $C_i \in \tilde{R}$ ensures that if any node in C_i is influenced by S , then all nodes in C_i are also influenced. For the entire collection $\tilde{\mathcal{R}}_t$, we define $\Omega_t(S, \tilde{\mathcal{R}}_t) = \sum_{\tilde{R} \in \tilde{\mathcal{R}}_t} \Omega_t(S, \tilde{R})$. Let $\tilde{\sigma}_t(S)$ denote the influence spread on the compressed graph D_t . Based on this, we construct an unbiased estimator $f_{\tilde{\mathcal{R}}_t}(S)$ for $\tilde{\sigma}_t(S)$:

$$f_{\tilde{\mathcal{R}}_t}(S) = \frac{\Omega_t(S, \tilde{\mathcal{R}}_t)}{|\tilde{\mathcal{R}}_t|} \cdot |V|, \quad (4)$$

We now establish that the estimator $f_{\tilde{\mathcal{R}}_t}(S)$, obtained through SCC-based sampling, serves as a concentration bound with a high probability, as demonstrated in Theorem 6.

Theorem 6. For any seed node set $S \subseteq V$ and user-defined parameters $\epsilon, \delta \in (0, 1)$, with probability at least $(1 - \delta)$, if $|\tilde{\mathcal{R}}_t| \geq \frac{(2+\epsilon) \cdot \log \delta}{\epsilon^2 \cdot \tilde{\sigma}_t(S)}$, then $f_{\tilde{\mathcal{R}}_t}(S)$ satisfies

$$|f_{\tilde{\mathcal{R}}_t}(S) - \tilde{\sigma}_t(S)| < \epsilon \cdot \tilde{\sigma}_t(S) \quad (5)$$

By the law of large numbers, as the number of samples increases, the estimation error of the expected influence decreases and stays within the ϵ -bound.

Influence estimation algorithm. Algorithm 4 presents our complete SCC-DAG based influence estimation framework. The framework first constructs the SCC partition and corresponding SCC-DAG for each snapshot, then performs sampling and estimation on the compressed graph structure. Since the SCC-DAG D_t is typically much smaller than the original snapshot G_t , our framework improves substantial computational efficiency while maintaining estimation accuracy. The theoretical guarantee is established by bounding the relative error of the estimation as follows.

Theoretical analysis. We begin by investigating how snapshot compression affects the influence function. Consider a snapshot $G_t(V, E_t)$ with a node partition $P_t = \{C_j\}_{j \in [q]}$ and its corresponding SCC-DAG $D_t(\Lambda_t, \Phi_t, p_t)$. We set the influence weights as follows: $\tilde{w}_t^{in}(u, v) = 1$ for any two nodes $u, v \in C_j$ within the same strongly connected component $C_j \in \Lambda_t$, and $\tilde{w}_t^{out}(u, v) = p_t(C_i, C_j)$ for nodes in different components where $u \in C_i$ and $v \in C_j$ with $i \neq j$. Given a seed set S , let $\tilde{\sigma}_t(S)$ and $\sigma_t(S)$ denote the influence spread on the compressed graph D_t and original graph G_t , respectively. We establish a lower bound for $\tilde{\sigma}_t(S)$ w.r.t. $\sigma_t(S)$.

Lemma 1. $\forall S \subseteq V, \sigma_t(S) \leq \tilde{\sigma}_t(S)$.

Then, we define the *strongly connected reliability* of each SCC-induced subgraph $G_t[C_j] = (V_j, E_j)$ as the probability that a randomly sampled subgraph of $G_t[C_j]$ is strongly connected [54]. Formally,

$$\text{Scr}(G_t[C_j]) = \sum_{X_j \subseteq E_j} \left(\prod_{e \in X_j} w_t(e) \cdot \prod_{e \in E_j \setminus X_j} (1 - w_t(e)) \cdot \kappa \right). \quad (6)$$

$\kappa = 1$ if the sampled subgraph (V_j, X_j) is strongly connected and $\kappa = 0$ otherwise. We now present an upper bound for $\tilde{\sigma}_t(S)$ using the strongly connected reliability of each component.

Lemma 2. $\forall S \subseteq V, \tilde{\sigma}_t(S) \leq \left(\prod_{j=1}^{\ell} \text{Scr}(G_t[C_j])^{-1} \right) \cdot \sigma_t(S)$.

By Theorem 6 and Lemmas 1 and 2, we have Theorem 7.

Theorem 7. *Given a snapshot graph G_t and a seed node set $S \subseteq V$, let $f_{\tilde{\mathcal{R}}_t}(S)$ denote the estimator of $\tilde{\sigma}_t(S)$. Then the relative error between $f_{\tilde{\mathcal{R}}_t}(S)$ and $\sigma_t(S)$ is bounded by:*

$$(1 - \epsilon) \cdot \sigma_t(S) \leq f_{\tilde{\mathcal{R}}_t}(S) \leq \left(\frac{1 + \epsilon}{\prod_{j=1}^{\ell} \text{Scr}(G_t[C_j])} \right) \cdot \sigma_t(S). \quad (7)$$

Time complexity. For each snapshot, the time complexity includes: (1) computing the SCC partition over r sampled graphs costs $O(r \cdot (|V| + |E_t|))$ time; (2) constructing the SCC-DAG takes $O(|E_t|)$ time; (3) sampling $\tilde{\mathcal{R}}_t$ costs $O(\mathcal{I}_t \cdot \frac{|\Phi_t|}{|\Lambda_t|} \cdot \sigma_t(C'))$, where C' is a random component selected by in-degree from D_t [36]. The overall time is $O(\mathcal{I}_t \cdot \frac{|\Phi_t|}{|\Lambda_t|} \cdot \sigma_t(C') + r \cdot (|V| + |E_t|))$, yielding a total complexity of $O(\sum_{t=1}^T (\mathcal{I}_t \cdot \frac{|\Phi_t|}{|\Lambda_t|} \cdot \sigma_t(C') + r \cdot (|V| + |E_t|)))$ across all snapshots.

VI. WIN-INFPM PROBLEM

In this section, we introduce a variant of the INFPM problem and present an effective method to address it.

In certain real-world scenarios such as product recommendations or public information campaigns, influence can

Algorithm 5 Prefix-Sum Sliding-Window (PssW)

Input: Temporal graph $\mathcal{G} = \{G_t\}_{t=1}^T$, a seed node set $\hat{S}$, threshold α , persistence θ , an α -valid snapshot set $t \in T_{\hat{S}}^{\alpha}$, and window size W

Output: Window-based influence persistence $F(\alpha, \theta, W, \hat{S})$

```

1: Initialize prefixSum  $\leftarrow \{0, 0, \dots, 0\}$ ;
2: for  $t = 0$  to  $T$  do //Build prefix sum array
3:   valid[t]  $\leftarrow \mathbf{1}[t \in T_{\hat{S}}^{\alpha}]$ ;
4:   prefixSum[t]  $\leftarrow$  prefixSum[t-1] + valid[t];
5:  $F(\alpha, \theta, W, \hat{S}) \leftarrow 0, \mathcal{T}_W(\hat{S}, \mathcal{G}) \leftarrow \emptyset, ts \leftarrow -1, te \leftarrow -1$ ;
6: for  $i = 1$  to  $T - W + 1$  do //Detect valid windows
7:   cnt  $\leftarrow$  prefixSum[i+W-1] - prefixSum[i-1];
8:   if cnt  $\geq \theta$  then
9:     if  $ts = -1 \wedge te = -1$  then  $ts \leftarrow i, te \leftarrow i + W - 1$ ;
10:    else if  $te + 1 = i + W - 1$  then  $te \leftarrow i + W - 1$ ;
11:    else if  $ts \neq -1 \wedge te \neq -1$  then
12:       $\mathcal{T}_W(\hat{S}, \mathcal{G}) \leftarrow \mathcal{T}_W(\hat{S}, \mathcal{G}) \cup \{[ts, te]\}, ts \leftarrow -1, te \leftarrow -1$ ;
13: if  $ts \neq -1 \wedge te \neq -1$  then  $\mathcal{T}_W(\hat{S}, \mathcal{G}) \leftarrow \mathcal{T}_W(\hat{S}, \mathcal{G}) \cup \{[ts, te]\}$ ;
14:  $F(\alpha, \theta, W, \hat{S}) \leftarrow \sum_{(t_h, t_t) \in \mathcal{T}_W(\hat{S}, \mathcal{G})} (t_t - t_h + 1)$ ;
15: Return  $F(\alpha, \theta, W, \hat{S})$ .
```

remain effective even with intermittent exposure, as long as the influence threshold is reached at multiple distinct timestamps within a given period. To capture such cases, we introduce a relaxed temporal influence model, which requires influence to be at least α at θ distinct timestamps within a larger window W (with $W > \theta$). Based on the above motivation, we define the concept of maximal (α, θ, W, S) -persistent influence interval.

Definition 5. (Maximal (α, θ, W, S) -persistent influence interval). *Given a temporal graph $\mathcal{G} = \{G_t\}_{t=1}^T$, a seed node set $S \subseteq V$, a time window $W > \theta$, and two parameters $\alpha \in (0, 1]$ and $\theta \geq 1$ ($\theta \in \mathbb{N}^+$), a time interval $[t_s, t_e] \subseteq [1, T]$ is a maximal (α, θ, S, W) -window persistent influence interval iff:*

- 1) **(α, θ, W) -persistent:** $\forall t \in [t_s, t_e], |\{t' \in [t, \min\{t+W-1, T\}] \mid \sigma_{t'}(S) \geq \alpha \cdot |V|\}| \geq \theta$.
- 2) **Maximal:** $\nexists [t'_s, t'_e] \supset [t_s, t_e]$ satisfies above condition.

Based on Definitions 2 and 5, we define $F(\alpha, \theta, W, S)$ as the window-based influence persistence of S in temporal graph $\mathcal{G}$, calculated as the total length of all maximal (α, θ, S, W) -persistent influence intervals. Then, we formally define the Window-based INFPM (WIN-INFPM) problem below.

Problem 2. (WIN-INFPM). *Given a temporal graph $\mathcal{G} = \{G_t\}_{t=1}^T$, a budget k , two parameters $\alpha \in (0, 1]$ and $\theta \geq 1$, and a time window $W > \theta$, WIN-INFPM aims to find a seed node set $\hat{S} \subseteq V$ with $|\hat{S}| = k$ such that the window-based influence persistence is maximized. Formally:*

$$\hat{S} = \arg \max_{\hat{S} \subseteq V \wedge |\hat{S}|=k} F(\alpha, \theta, W, \hat{S}). \quad (8)$$

To address the WIN-INFPM problem, we introduce a new evaluation procedure, Prefix-Sum Sliding-Window (PssW), into the RevG and LRep frameworks. PssW efficiently computes the window-based influence persistence $F(\alpha, \theta, W, \hat{S})$ for a given seed set $\hat{S}$, as shown in Algorithm 5. PssW first builds a prefix sum array by checking whether each timestamp t is α -valid, i.e., whether the influence spread of $\hat{S}$ in G_t is at least α , and cumulatively counts these valid timestamps (Lines 1-4). It then slides a window of length W over time, using prefix sum subtraction to efficiently compute the number of α -valid timestamps in each window (Line 7). A window is marked as persistent if it contains at least θ α -valid timestamps. Each persistent window is extended

TABLE I: Dataset statistics

Dataset	$ V $	$ E $	T	Time scale
<i>CollegeMsg</i> (CM)	1,899	59,835	193	Day
<i>EmailCore</i> (EC)	986	332,334	803	Day
<i>MathOverflow</i> (MO)	24,818	506,550	2,350	Day
<i>AskUbuntu</i> (AU)	159,316	964,437	2,613	Day
<i>SuperUser</i> (SU)	194,085	1,443,339	2,773	Day
<i>WikiTalk</i> (WT)	1,140,149	7,833,140	696	Day
<i>StackOverflow</i> (SO)	2,601,977	63,497,050	277	Day

to include adjacent windows satisfying the same condition, forming maximal intervals stored in $\mathcal{T}_W(\hat{S}, \mathcal{G})$ (Lines 5-13). Finally, the influence persistence is returned (Lines 14-15).

Time Complexity. Building the prefix sum array over T snapshots takes $O(T)$ time. Scanning the $T - W + 1$ sliding windows and aggregating valid intervals also take $O(T)$ time in the worst case. Thus, the overall time complexity is $O(T)$.

VII. EXPERIMENTS

In this section, we empirically evaluate our proposed algorithms. All experiments are implemented in C++ on a Linux machine with a 3.70GHz CPU and 128GB of RAM. The code is available at <https://github.com/ZJU-DAILY/INFPM>.

Datasets. We conduct experiments on seven real-world temporal social networks, all publicly available from SNAP [55]. Dataset statistics are summarized in Table I. For each network, we adopt the *weighted-cascade* model [1] to assign influence weights $w_t(u, v)$ on edges in each snapshot G_t , where $w_t(u, v) = 1/|N_t^{in}(v)|$, following prior work [8], [45].

Algorithms. We compare a set of algorithms in experiments.

- **RevG** and **LRep**: ReverseGreedy and LazyReplace algorithms that utilize the reverse influence sampling (RIS) method [34] to estimate influence spread.
- **RevG+** and **LRep+**: ReverseGreedy and LazyReplace algorithms that employ the influence estimation framework proposed in Section V.
- **Influential (INF)** [56]: A method that selects seeds with the highest average influence across all snapshots.
- **HighDegree (DEG)** [40]: A method that selects seeds with the highest average out-degrees across all snapshots.
- **Random (RAN)**: A method that randomly selects seeds.
- **Enumerate (Enum)**: An exact method that enumerate all size- k seed sets, computes their influence persistence, and returns the one with the maximum influence persistence.

Parameter and Metric Settings. We evaluate the performance of algorithms across different parameters, including the budget k , influence threshold α , persistence threshold θ , the number of samples per snapshot $\mathcal{I}_t$, and time span T . In each experiment, only one parameter is varied while the others are fixed at their default values: $\theta = 2$ for all datasets; the number of samples per snapshot is set to 6×10^3 for the RIS-based approach and 3×10^3 for SCC-DAG-based method based on the theoretical analysis; T is set to the entire time scale of the graph; k and α are set differently for different datasets. Notably, the maximum value of α is set as the ratio of the average number of non-isolated nodes across all snapshots to the total number of nodes, where a node is non-isolated if it has at least one interaction in the snapshot. We ensure that $k < \alpha \cdot |V|$ to prevent all

TABLE II: Exact VS. Approximate on EC-Dept3

Parameter	$k = 3$			$k = 4$		
	Enum	RevG	LRep	Enum	RevG	LRep
Influence persistence	31	24	28	74	63	71
Running time (sec)	444.06	0.6014	0.6308	13846	0.6186	0.6417
Approximation ratio	/	77.4%	90.3%	/	85.1%	95.9%
Parameter	$k = 5$			$k = 6$		
	Enum	RevG	LRep	Enum	RevG	LRep
Influence persistence	142	125	128	/	198	219
Running time (sec)	305722	0.6235	0.6674	> 7 days	0.6241	0.6901
Approximation ratio	/	88.0%	90.1%	/	/	/

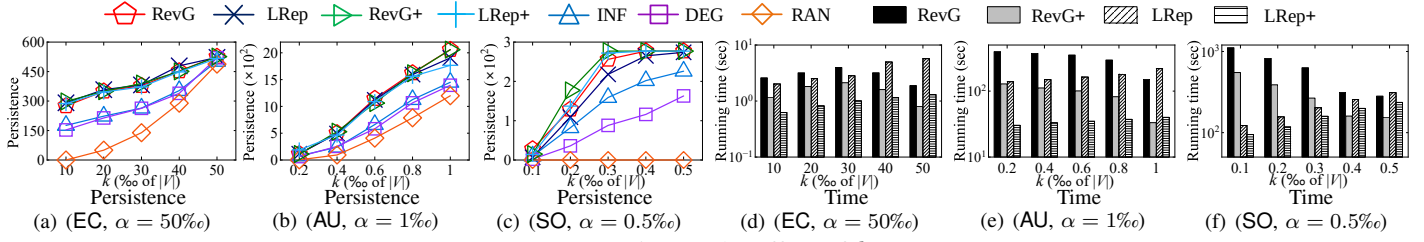
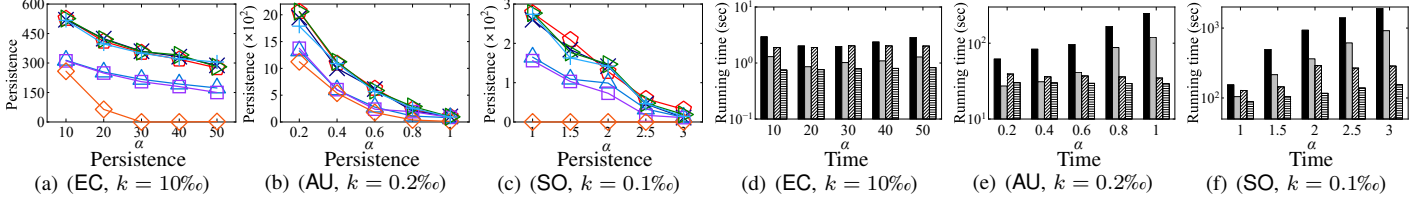
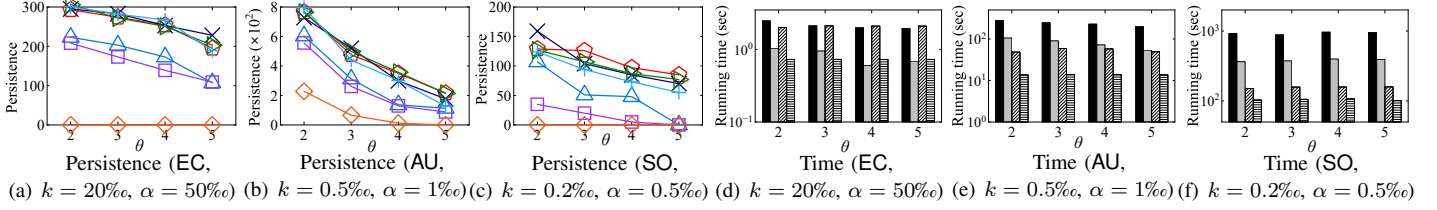
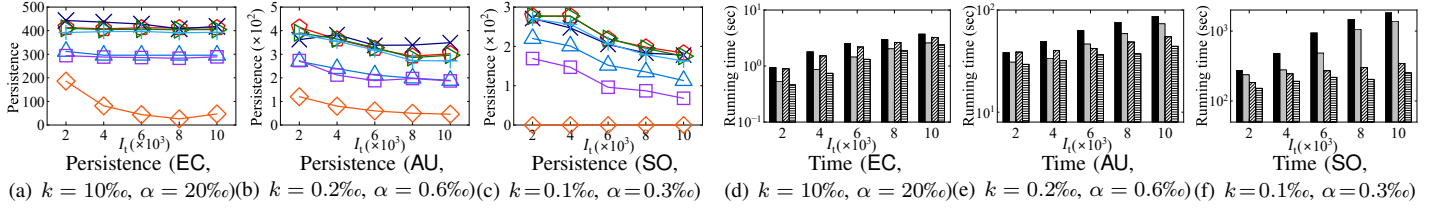
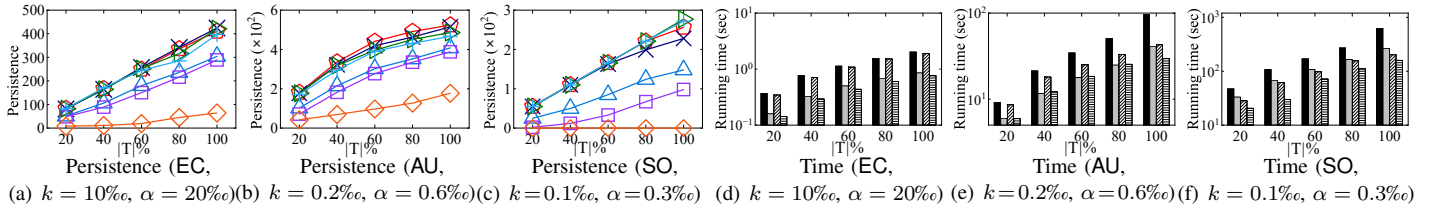
snapshots from automatically satisfying the α -condition. We report the influence persistence and running time (second) for each experiment. In all experiments, influence spread on each snapshot is estimated using 10^5 Monte Carlo simulations, and each algorithm is run five times with average results reported.

A. Evaluation of INFPM Algorithms

This subsection evaluates the INFPM algorithms. Due to space constraints and similar results across datasets, we report results for EC, AU, and SO in this paper. Results for all datasets are provided in the Appendix.

Exp-1: Exact VS. Approximate. Due to the NP-hardness of INFPM, we propose approximate algorithms in Section IV. In this experiment, we compare the exact algorithm Enum with our proposed approximate algorithms RevG and LRep over a small temporal graph EC-Dept3 [55] ($|V| = 89$, $|E| = 12, 216$, $T = 803$), with $\alpha = 10\%$ and $\theta = 2$. Table II presents the results. We observe that RevG and LRep achieve influence persistence values close to that of Enum, with average approximation ratios of 83.5% and 92.1%, respectively. In terms of efficiency, both RevG and LRep are 3-10 orders of magnitude faster than Enum. Notably, Enum failed to complete within a week even for $k = 6$, highlighting the practical necessity of approximate algorithms.

Exp-2: Effect of k . We investigate the impact of the number of seed nodes k (set as % of $|V|$), with results shown in Figure 7. In Figures 7(a)-7(c), RevG, LRep, RevG+, and LRep+ consistently achieve the highest influence persistence, validating the effectiveness of our proposed algorithms. Specifically, compared to the optimal baseline INF, our algorithms achieve up to 100% improvement in total influence persistence. Moreover, influence persistence increases with larger k , as more seeds influence more nodes per snapshot, raising the number of snapshots satisfying threshold α . Figures 7(d)-7(f) show that RevG+ and LRep+ consistently outperform RevG and LRep, benefiting from SCC-DAG-based sampling on compressed graphs, which shortens sampling paths and reduces node visits per SCC-RR set, leading to up to a 400% improvement in efficiency over RevG and LRep. Additionally, as k increases, RevG and RevG+ generally run faster, since the greedy deletion process involves fewer iterations. In contrast, LRep and LRep+ exhibit increased running time with larger k , due to larger initial seed set and more replacements in each iteration. Furthermore, LRep (resp. LRep+) is generally faster than RevG (resp. RevG+) in most cases. This is because LRep (resp. LRep+) prioritizes replacing the weakest seed and terminates early if no improvement in influence persistence

Fig. 7: The effect of k Fig. 8: The effect of α Fig. 9: The effect of θ Fig. 10: The effect of I_t Fig. 11: The effect of T

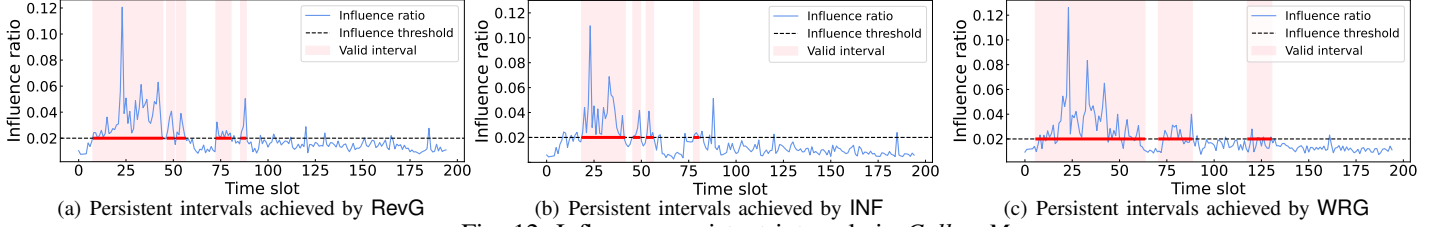
is observed. In contrast, RevG (resp. RevG+) exhaustively examines all candidates in each iteration to select one for removal, resulting in a higher computational cost per iteration.

Exp-3: Effect of α . We study the impact of the influence threshold α and report the results in Figure 8. Figures 8(a)-8(c) reveal that the influence persistence decreases as α increases. This occurs because, for a fix-sized seed set, a larger α results in fewer snapshots meeting the threshold, thereby reducing influence persistence. Figures 8(d)-8(f) show that the running times of LRep and LRep+ remain relatively stable on EC and AU, as their runtime is dominated by seed replacement, which primarily depends on the seed budget k . Varying α only affects the computation of persistent intervals, which incurs relatively minor overhead. However, on the larger graph SO, the running times of LRep and LRep+ increase as α grows. This is because higher α values lead to fewer qualifying snapshots, which become more sparsely and discontinuously distributed. This increases the randomness in whether replacing the weakest node improves the influence persistence, thereby leading to

more replacements and persistence evaluations. In contrast, the running times of RevG and RevG+ increase with α , because when the candidate set size exceeds $\alpha \cdot |V|$, we retain $\alpha \cdot |V|$ candidates. Thus, a larger α results in more candidates being preserved and processed, increasing the total running time.

Exp-4: Effect of θ . We explore the impact of the persistence threshold θ , with results shown in Figure 9. In Figures 9(a)-9(c), influence persistence decreases as θ grows. This is because, given a fixed set of snapshots meeting threshold α , a higher θ enforces stricter continuity, yielding fewer qualified time intervals and lower overall influence persistence. Figures 9(d)-9(f) show that the running times of LRep and LRep+ remain nearly constant, while those of RevG and RevG+ slightly decrease as θ increases. This occurs because larger θ values cause more snapshots to be pruned during candidate set generation, shrinking the candidate set and reducing the time needed for greedy node deletion, thus lowering total runtime.

Exp-5: Effect of I_t . We evaluate the impact of the number of samples per snapshot I_t . The results are presented in

Fig. 12: Influence persistent intervals in *CollegeMsg*TABLE III: Influence persistence vs. W ($\theta = 5$)

Dataset	CM		EC		MO		AU		SU		WT		SO	
Alg.	WRG	WLR	WRG	WLR	WRG	WLR	WRG	WLR	WRG	WLR	WRG	WLR	WRG	WLR
$W = 6$	34	28	427	423	413	516	328	362	657	491	818	816	224	210
$W = 7$	38	36	481	476	633	845	565	701	946	1138	995	941	316	256
$W = 8$	47	32	532	549	969	996	1004	984	1537	1330	1032	934	422	310
$W = 9$	49	52	572	561	1225	1280	1327	1246	1647	1643	1053	965	516	387
$W = 10$	53	51	520	536	1460	1560	1739	1484	2277	2194	1105	1004	571	502

TABLE IV: Running time (sec) vs. W ($\theta = 5$)

Dataset	CM		EC		MO		AU		SU		WT		SO	
Alg.	WRG	WLR	WRG	WLR	WRG	WLR	WRG	WLR	WRG	WLR	WRG	WLR	WRG	WLR
$W = 6$	1.011	0.670	2.512	2.035	42.94	16.14	268.9	51.02	544.7	85.91	297.4	264.5	762.8	130.9
$W = 7$	0.885	0.675	2.601	2.111	41.23	17.83	268.9	50.74	560.8	83.8	215.1	206.5	593.3	131.2
$W = 8$	0.756	0.668	3.029	2.377	37.23	16.89	268.6	47.56	551.9	78.06	151.7	213.2	484.8	204.4
$W = 9$	0.911	0.727	3.018	2.146	41.47	15.55	271.6	45.67	549.1	83.84	156.7	202.9	397.6	217.3
$W = 10$	0.888	0.671	2.977	2.134	41.35	16.61	242.7	47.65	484.8	84.47	114.6	179.5	355.7	211.2

Figure 10. In Figures 10(a)-10(c), it is observed that as $\mathcal{I}_t$ increases, the influence persistence of all algorithms initially decreases and then stabilizes. This is because a larger $\mathcal{I}_t$ enables a more accurate unbiased estimation of influence spread within each snapshot, thereby yielding more consistent influence persistence, as proven by Theorem 6. Furthermore, as shown in Figures 10(d)-10(f), the running times of all algorithms increase with growing $\mathcal{I}_t$.

Exp-6: Effect of T . We study the scalability of our algorithms by varying the time span T , i.e., the number of snapshots. To this end, we retain different fractions of snapshots from the entire temporal graph to generate temporal graphs with different T s. The results are reported in Figure 11. As expected, both the influence persistence and running time increase when the time span grows. This is because more snapshots lead to more qualified persistent intervals and longer processing time for computing the influence spread across all snapshots.

B. Evaluation of WIN-INFPM Algorithms

In this section, we assess the performance of the WRG and WLR algorithms for the WIN-INFPM problem. Derived from RevG and LRep respectively, both algorithms integrate PssW procedure to compute window-based influence persistence.

Exp-7: Effect of W . We evaluate the effect of the window size W while fixing the persistence threshold at the default value $\theta = 5$. As shown in Table III, influence persistence for WRG and WLR increases as W grows. The reason behind is that, a larger window size enhances the temporal continuity flexibility, allowing snapshots that exceed the influence threshold and occur close in time, even if not strictly consecutive, to be counted together within a persistent interval. Consequently, a greater number of intervals satisfy the persistence condition, and longer qualifying intervals are more likely to emerge, improving the overall influence persistence. Moreover, Table IV shows the running time of WRG and WLR w.r.t. different W . On smaller datasets, both algorithms exhibit relatively stable runtime with minimal sensitivity to window size. However, on larger datasets, both WRG and WLR run faster when W increases. This is because the increased window size produces fewer but longer valid intervals, which can be constructed through simple extensions, lowering the overhead of re-evaluating each timestamp for interval validity.

C. Case Study

We conduct a case study on the real-world temporal social network *CollegeMsg*, which records message exchanges among university users over time. We consider scenarios such as campus-wide event promotion or sustained visibility of safety alerts, where maintaining long-term user engagement is the key objective. To model this, we define a persistent interval as a period during which the number of influenced users exceeds a threshold (i.e., $\alpha = 20\%$) for at least $\theta = 3$ consecutive days. To this end, the goal is to select a small set of seed users (i.e., $k = 10\%$) to maximize the total duration of persistent intervals. We visualize three cases to illustrate persistent intervals under different actions taken by the college.

Figure 12(a) shows the persistent intervals achieved by RevG, where the influence threshold is exceeded for at least 3 consecutive days (i.e., INFPM). Most time slots with over 20% influenced users fall within the 0–100 range, yielding several durations that satisfy the consecutive requirement. Due to space constraints, we only present results for RevG, as LRep yields similar outcomes. Figure 12(b) presents the persistent intervals produced by the baseline INF method. Compared to RevG, INF produces fewer and shorter persistent intervals, leading to lower overall influence persistence. Figure 12(c) displays the persistent intervals identified by WRG under the condition that at least 3 days within each one-week window ($W = 7$) reach more than 20% of users (i.e., WIN-INFPM). This added flexibility enables WRG to include time slots that individually fall short of the threshold but collectively contribute to sustained influence within the window, yielding longer intervals and higher influence persistence than RevG.

VIII. CONCLUSION

In this paper, we introduce the novel Influence Persistence Maximization (INFPM) problem. Given the NP-hardness of INFPM, we propose two seed selection algorithms: RevG, which employs a reverse deletion greedy strategy, and LRep, which utilizes a replacement-based strategy. To improve the efficiency of RevG and LRep, we propose a new influence computation method. Furthermore, we study a practical variant WIN-INFPM. Extensive experiments demonstrate that our proposed algorithms are effective, efficient, and scalable.

REFERENCES

- [1] D. Kempe, J. Kleinberg, and É. Tardos, "Maximizing the spread of influence through a social network," in *Proceedings of the 9th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2003, pp. 137–146.
- [2] P. Domingos and M. Richardson, "Mining the network value of customers," in *Proceedings of the 7th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2001, pp. 57–66.
- [3] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen, and N. Glance, "Cost-effective outbreak detection in networks," in *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2007, pp. 420–429.
- [4] M. Ye, X. Liu, and W.-C. Lee, "Exploring social influence for recommendation: a generative model approach," in *Proceedings of the 35th international ACM SIGIR conference on Research and development in information retrieval*, 2012, pp. 671–680.
- [5] Y. Tang, Y. Shi, and X. Xiao, "Influence maximization in near-linear time: A martingale approach," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 2015, pp. 1539–1554.
- [6] J. Tang, X. Tang, X. Xiao, and J. Yuan, "Online processing algorithms for influence maximization," in *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*, 2018, pp. 991–1005.
- [7] Q. Guo, C. Feng, F. Zhang, and S. Wang, "Efficient algorithm for budgeted adaptive influence maximization: An incremental rr-set update approach," *Proceedings of the ACM on Management of Data*, vol. 1, no. 3, pp. 1–26, 2023.
- [8] Y. Huang, S. Zhang, L. V. Lakshmanan, W. Lin, X. Xiao, and B. Tang, "Efficient and effective algorithms for a family of influence maximization problems with a matroid constraint," *Proceedings of the VLDB Endowment*, vol. 18, no. 2, pp. 117–129, 2024.
- [9] C. Feng, X. Chen, Q. Guo, F. Zhang, and S. Wang, "Efficient approximation algorithms for minimum cost seed selection with probabilistic coverage guarantee," *Proceedings of the ACM on Management of Data*, vol. 2, no. 4, pp. 1–26, 2024.
- [10] J. Leskovec, J. Kleinberg, and C. Faloutsos, "Graphs over time: densification laws, shrinking diameters and possible explanations," in *Proceedings of the 7th ACM SIGKDD international conference on Knowledge discovery in data mining*, 2005, pp. 177–187.
- [11] J. Leskovec, L. Backstrom, R. Kumar, and A. Tomkins, "Microscopic evolution of social networks," in *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2008, pp. 462–470.
- [12] K. Everywhere, "55 facebook stats for your social media strategy [2024]." 2024. [Online]. Available: <https://keywordseverywhere.com/blog/facebook-stats/>
- [13] Y. Yang and J. Pei, "Influence analysis in evolving networks: A survey," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 33, no. 3, pp. 1045–1063, 2019.
- [14] J. Zhao, S. Shang, P. Wang, J. C. Lui, and X. Zhang, "Tracking influential nodes in time-decaying dynamic interaction networks," in *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 2019, pp. 1106–1117.
- [15] J. Zhao, P. Wang, W. Zhang, Z. Zhang, M. Liu, J. Tao, and J. C. Lui, "Tracking influencers in decaying social activity streams with theoretical guarantees," *IEEE/ACM Transactions on Networking*, vol. 32, no. 2, pp. 1461–1476, 2023.
- [16] Y. Wang, Q. Fan, Y. Li, and K.-L. Tan, "Real-time influence maximization on dynamic social streams," *Proceedings of the VLDB Endowment*, vol. 10, no. 7, pp. 805–816, 2017.
- [17] W. Li, Q. Bai, M. Zhang, and T. D. Nguyen, "Automated influence maintenance in social networks: an agent-based approach," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 31, no. 10, pp. 1884–1897, 2018.
- [18] X. Wu, L. Fu, Z. Zhang, H. Long, J. Meng, X. Wang, and G. Chen, "Evolving influence maximization in evolving networks," *ACM Transactions on Internet Technology (TOIT)*, vol. 20, no. 4, pp. 1–31, 2020.
- [19] N. Ohsaka, T. Akiba, Y. Yoshida, and K.-i. Kawarabayashi, "Dynamic influence analysis in evolving networks," *Proceedings of the VLDB Endowment*, vol. 9, no. 12, pp. 1077–1088, 2016.
- [20] G. Song, Y. Li, X. Chen, X. He, and J. Tang, "Influential node tracking on dynamic social network: An interchange greedy approach," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 29, no. 2, pp. 359–372, 2016.
- [21] Y. Yang, Z. Wang, J. Pei, and E. Chen, "Tracking influential individuals in dynamic networks," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 29, no. 11, pp. 2615–2628, 2017.
- [22] Y. Yang, Z. Wang, T. Jin, J. Pei, and E. Chen, "Tracking top-k influential users with relative errors," in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, 2019, pp. 1783–1792.
- [23] V. K. Yalavarthi and A. Khan, "Steering top-k influencers in dynamic graphs via local updates," in *2018 IEEE International Conference on Big Data (Big Data)*. IEEE, 2018, pp. 576–583.
- [24] K. Kovel, "Short-term vs. long-term digital marketing campaigns." 2025. [Online]. Available: <https://thezoeteam.com/blog/short-term-vs-long-term-marketing>
- [25] Builtin, "How the mere exposure effect works in marketing and advertising." 2023. [Online]. Available: <https://builtin.com/articles/mere-exposure-effect>
- [26] B. Moritz, "The strategic use of repeated exposure and the phenomenon of familiarity." 2024. [Online]. Available: <https://www.mxmoritz.com/article/the-mere-exposure-effect-in-marketing-and-sales>
- [27] B. Denison, H. Dahlen, J.-E. C. Kim, C. Williams, E. Kranzler, J. N. Luchman, S. Trigger, M. Bennett, T. Nighbor, M. Vines et al., "Evaluation of the 'we can do this' campaign paid media and covid-19 vaccination uptake, united states, december 2020–january 2022," *Journal of health communication*, vol. 28, no. 9, pp. 573–584, 2023.
- [28] Shopify, "Influencer pricing: The cost of influencers in 2025." 2025. [Online]. Available: <https://www.shopify.com/sg/blog/influencer-pricing>
- [29] D. C. . Extension, "The role of diffusion networks and critical mass in adoption." 2022. [Online]. Available: <https://csr.education/development-communication-extension/diffusion-networks-critical-mass-adoption/>
- [30] M. Elsen, R. Pieters, and M. Wedel, "Effects of advertising exposure duration and frequency: a theory and initial test," *Journal of Marketing Analytics*, pp. 1–19, 2025.
- [31] Later, "Instagram influencer marketing: Master guide for brands in 2025." 2025. [Online]. Available: <https://later.com/blog/instagram-influencer-marketing/>
- [32] Openinfluence, "Youtube influencer marketing." 2025. [Online]. Available: <https://openinfluence.com/youtube-influencer-marketing/>
- [33] L. Zou, A. Ceria, and H. Wang, "Short-and long-term temporal network prediction based on network memory," *Applied Network Science*, vol. 8, no. 1, p. 76, 2023.
- [34] C. Borgs, M. Brautbar, J. Chayes, and B. Lucier, "Maximizing social influence in nearly optimal time," in *Proceedings of the twenty-fifth annual ACM-SIAM symposium on Discrete algorithms*. SIAM, 2014, pp. 946–957.
- [35] K. Huang, S. Wang, G. Bevilacqua, X. Xiao, and L. V. Lakshmanan, "Revisiting the stop-and-stare algorithms for influence maximization," *Proceedings of the VLDB Endowment*, vol. 10, no. 9, pp. 913–924, 2017.
- [36] Y. Tang, X. Xiao, and Y. Shi, "Influence maximization: Near-optimal time complexity meets practical efficiency," in *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, 2014, pp. 75–86.
- [37] Q. Guo, S. Wang, Z. Wei, W. Lin, and J. Tang, "Influence maximization revisited: Efficient sampling with bound tightened," *ACM Transactions on Database Systems (TODS)*, 2022.
- [38] K. Han, B. Wu, J. Tang, S. Cui, C. Aslay, and L. V. Lakshmanan, "Efficient and effective algorithms for revenue maximization in social advertising," in *Proceedings of the 2021 ACM SIGMOD International Conference on Management of Data*, 2021, pp. 671–684.
- [39] P. Banerjee, W. Chen, and L. V. Lakshmanan, "Maximizing welfare in social networks under a utility driven influence diffusion model," in *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data*, 2019, pp. 1078–1095.
- [40] J. Tang, X. Tang, and J. Yuan, "Profit maximization for viral marketing in online social networks: Algorithms and analysis," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 30, no. 6, pp. 1095–1108, 2017.
- [41] X. Chang, X. Ke, L. Chen, C. Ge, Z. Wei, and Y. Gao, "Host profit maximization: Leveraging performance incentives and user flexibility," *Proceedings of the VLDB Endowment*, vol. 17, no. 1, pp. 51–64, 2023.
- [42] X. Chang, Q. Liu, Y. Gao, B. Zheng, Y. Cai, and Q. Li, "The most influenced community search on social networks," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 2025, pp. 2601–2614.
- [43] X. Chang, J. Fu, Q. Liu, Y. Gao, and B. Zheng, "Time-aware influence minimization via blocking social networks," in *2025 IEEE 41st Inter-*

national Conference on Data Engineering (ICDE). IEEE Computer Society, 2025, pp. 557–570.

- [44] J. Xie, Z. Chen, D. Chu, F. Zhang, X. Lin, and Z. Tian, “Influence maximization via vertex countering,” *Proceedings of the VLDB Endowment*, vol. 17, no. 6, pp. 1297–1309, 2024.
- [45] J. Wang, Y. Wu, X. Wang, Y. Zhang, L. Qin, W. Zhang, and X. Lin, “Efficient influence minimization via node blocking,” *Proceedings of the VLDB Endowment*, vol. 17, no. 10, pp. 2501–2513, 2024.
- [46] Y. Li, J. Fan, Y. Wang, and K.-L. Tan, “Influence maximization on social graphs: A survey,” *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 30, no. 10, pp. 1852–1872, 2018.
- [47] Z. Liang, Y. Yang, X. Ke, X. Xiao, and Y. Gao, “A benchmark study of deep-rl methods for maximum coverage problems over graphs,” *Proceedings of the VLDB Endowment*, vol. 17, no. 11, pp. 3666–3679, 2024.
- [48] Z. Aghaee, M. M. Ghasemi, H. A. Beni, A. Bouyer, and A. Fatemi, “A survey on meta-heuristic algorithms for the influence maximization problem in the social networks,” *Computing*, vol. 103, pp. 2437–2477, 2021.
- [49] S. Huang, Z. Bao, J. S. Culpepper, and B. Zhang, “Finding temporal influential users over evolving social networks,” in *2019 IEEE 35th international conference on data engineering (ICDE)*. IEEE, 2019, pp. 398–409.
- [50] R.-H. Li, J. Su, L. Qin, J. X. Yu, and Q. Dai, “Persistent community search in temporal networks,” in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 797–808.
- [51] W. Chen, C. Wang, and Y. Wang, “Scalable influence maximization for prevalent viral marketing in large-scale social networks,” in *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2010, pp. 1029–1038.
- [52] M. O. Ball, “Complexity of network reliability computations,” *Networks*, vol. 10, no. 2, pp. 153–165, 1980.
- [53] L. G. Valiant, “The complexity of enumeration and reliability problems,” *siam Journal on Computing*, vol. 8, no. 3, pp. 410–421, 1979.
- [54] N. Ohsaka, T. Sonobe, S. Fujita, and K.-i. Kawarabayashi, “Coarsening massive influence networks for scalable diffusion analysis,” in *Proceedings of the 2017 ACM SIGMOD international conference on management of data*, 2017, pp. 635–650.
- [55] J. Leskovec and A. Krevl, “SNAP Datasets: Stanford large network dataset collection,” <http://snap.stanford.edu/data>, Jun. 2014.
- [56] M. Simpson, F. Hashemi, and L. V. Lakshmanan, “Misinformation mitigation under differential propagation rates and temporal penalties,” *Proceedings of the VLDB Endowment*, vol. 15, no. 10, pp. 2216–2229, 2022.
- [57] M. Mitzenmacher and E. Upfal, *Probability and computing: Randomization and probabilistic techniques in algorithms and data analysis*. Cambridge university press, 2017.



Baihua Zheng received the PhD degree in computer science from the Hong Kong University of Science & Technology, China, in 2003. She is currently a professor with the School of Computing and Information Systems, Singapore Management University, Singapore. Her research interests include mobile/pervasive computing, spatial databases, and Big Data analytics.



Yunjun Gao (Senior Member, IEEE) received the PhD degree in computer science from Zhejiang University, China, in 2008. He is currently a professor with the College of Computer Science, Zhejiang University, China. His research interests include database, Big Data management and analytics, and AI interaction with DB technology.



Xueqin Chang received the BS degree in computer science from Xidian University, China, in 2021. She is currently working toward the PhD degree with Zhejiang University, China. Her research interests include social network influence analysis and graph analytics.



Qing Liu received PhD degree from the College of Computer Science, Zhejiang University, China. He is currently a professor with the College of Computer Science, Zhejiang University, China. His research interests include graph management, database usability, and secure graph analysis.

APPENDIX A PROOF OF THEOREM 3

Proof. We prove Theorem 3 by contradiction. Suppose snapshot G_t cannot be excluded, implying there exists an optimal seed set S^* that needs G_t to achieve maximum influence persistence. For G_t to contribute to the persistence of S^* , we must have $\sigma_t(S^*) \geq \alpha \cdot |V|$ (by the α -valid condition). However, the theorem premise states that $\max_{S \subseteq V, |S|=k} \sigma_t(S) < \alpha \cdot |V|$, which means $\sigma_t(S^*) < \alpha \cdot |V|$. This contradicts the α -valid requirement. Therefore, G_t can be safely excluded without affecting optimality. $\square$

APPENDIX B PROOF OF THEOREM 4

Proof. We prove Theorem 4 by contradiction. Suppose snapshot G_t with $t \notin T_\theta$ cannot be excluded, implying there exists an optimal seed set S^* that requires G_t to achieve maximum influence persistence. For G_t to contribute to the persistence of S^* , it must be part of a consecutive temporal interval of length at least θ (by the θ -persistent condition). However, by definition of T_θ , snapshot G_t is not contained in any persistent influence interval of length $\geq \theta$ when applying GetPersistence to the α -valid snapshots T_α . This means G_t cannot satisfy the θ -persistent requirement for any feasible solution. Thus, G_t can be safely excluded without affecting optimality. $\square$

APPENDIX C PROOF OF THEOREM 5

Proof. We prove that if a timestamp $t \notin \bigcup_{I \in \mathcal{T}(A, \mathcal{G})} I$, then for any $A' \subset A$, $t \notin \bigcup_{I' \in \mathcal{T}(A', \mathcal{G})} I'$. Suppose for contradiction that $\exists A' \subset A$ such that $t \in \bigcup_{I' \in \mathcal{T}(A', \mathcal{G})} I'$. Then $\exists I' = [t_s, t_e] \in \mathcal{T}(A', \mathcal{G})$ with $t \in I'$ that is a maximal (α, θ, A') -persistent influence interval. Since $A' \subset A$ and $\sigma_t(\cdot)$ is monotonic, we have $\sigma_{t'}(A) \geq \sigma_{t'}(A') \geq \alpha \cdot |V|$ for all $t' \in I'$. Thus I' satisfies both α -valid and θ -persistent for A . By Definition 1, I' must be contained in some maximal (α, θ, A) -persistent influence interval $I \in \mathcal{T}(A, \mathcal{G})$. Since $t \in I' \subseteq I$, we have $t \in \bigcup_{I \in \mathcal{T}(A, \mathcal{G})} I$, contradicting our assumption. Therefore, $\forall A' \subset A$, $t \notin \bigcup_{I' \in \mathcal{T}(A', \mathcal{G})} I'$. $\square$

APPENDIX D PROOF OF THEOREM 6

Proof. We prove it by extending chernoff inequalities [57]. Applying chernoff inequalities, we have $\Pr[f_{\tilde{\mathcal{R}}_t}(S) \cdot |\tilde{\mathcal{R}}_t| - |\tilde{\mathcal{R}}_t| \cdot \tilde{\sigma}_t(S) \geq \epsilon \cdot |\tilde{\mathcal{R}}_t| \cdot \tilde{\sigma}_t(S)] \leq \exp\left(-\frac{\epsilon^2}{2+\epsilon} \cdot |\tilde{\mathcal{R}}_t| \cdot \tilde{\sigma}_t(S)\right)$. When $|\tilde{\mathcal{R}}_t| \geq \frac{(2+\epsilon) \cdot \log \delta}{\epsilon^2 \cdot \tilde{\sigma}_t(S)}$, we have $\Pr[f_{\tilde{\mathcal{R}}_t}(S) - \tilde{\sigma}_t(S) \geq \epsilon \cdot \tilde{\sigma}_t(S)] \leq \delta$. Thus, $|f_{\tilde{\mathcal{R}}_t}(S) - \tilde{\sigma}_t(S)| < \epsilon \cdot \tilde{\sigma}_t(S)$ holds with probability at least $1 - \delta$. $\square$

APPENDIX E PROOF OF LEMMA 1

Proof. In the compressed graph D_t , nodes within the same SCC C_j can influence each other with weight $\tilde{w}_t^{in}(u, v) = 1$, while inter-component influence is given by $w_t^{out}(u, v) =$

$p_t(C_i, C_j)$ for $u \in C_i, v \in C_j$. Then, we have $\tilde{w}_t^{in}(u, v) = 1 \geq w_t(u, v)$ and $w_t^{out}(u, v) = p_t(C_i, C_j) = 1 - \prod_{(u, v) \in E_t, u \in C_i, v \in C_j} (1 - w_t(u, v)) \geq w_t(u, v)$. Compared to the original graph G_t , where influence follow actual paths and edge activations, D_t assumes maximal intra-SCC influence and abstracts inter-SCC influence probabilistically. Therefore, for any seed set $S \subseteq V$, the estimated influence $\tilde{\sigma}_t(S)$ in D_t is at least the true influence $\sigma_t(S)$ in G_t . $\square$

APPENDIX F PROOF OF LEMMA 2

Proof. Let the edge set E_t be partitioned into disjoint subsets $E_t^1, \dots, E_t^\ell$ and $E_t^0 = E_t \setminus \bigcup_{j=1}^\ell E_t^j$, where each E_t^j contains all intra-cluster edges within node subset $C_j \subseteq V$, and E_t^0 contains the remaining inter-cluster edges. For each $j \in [\ell]$, define the strongly connected reliability of the subgraph $G_t[C_j]$ as: $\text{Scr}(G_t[C_j]) = \sum_{X \subseteq E_t^j} p(X | E_t^j) \cdot \mathbf{1}[(C_j, X) \text{ is strongly connected (SC)}]$ [54]. Let $\mathcal{E}_j \subseteq E_t^j$ and $\mathcal{E}_0 \subseteq E_t^0$ be the random edge subsets sampled under the probabilistic model. Define $\mathcal{E} = \bigcup_{j=1}^\ell \mathcal{E}_j \cup \mathcal{E}_0$ and $E'_t = \bigcup_{j=1}^\ell E_t^j \cup \mathcal{E}_0$. $r(V, \mathcal{E})(S) = r(V, E'_t)(S)$ when every subgraph $(C_j, \mathcal{E}_j)$ is strongly connected for all $j \in [\ell]$. Thus,

$$\begin{aligned} \sigma_t(S) &= \sum_{\mathcal{E}_1 \subseteq E_1} \cdots \sum_{\mathcal{E}_\ell \subseteq E_\ell} \left(\prod_{j=1}^\ell p(\mathcal{E}_j | E_t^j) \right) \sum_{\mathcal{E}_0 \subseteq E_0} p(\mathcal{E}_0 | E_t^0) \cdot r(V, \mathcal{E})(S) \\ &\geq \sum_{\mathcal{E}_1 \subseteq E_1} \cdots \sum_{\mathcal{E}_\ell \subseteq E_\ell} \left(\prod_{j=1}^\ell p(\mathcal{E}_j | E_t^j) \cdot \mathbf{1}[(C_j, \mathcal{E}_j) \text{ is SC}] \right) \\ &\quad \cdot \sum_{\mathcal{E}_0 \subseteq E_0} p(\mathcal{E}_0 | E_t^0) \cdot r(V, E'_t)(S) \\ &= \left(\prod_{j=1}^\ell \text{Scr}(G_t[C_j]) \right) \cdot \tilde{\sigma}_t(S) \end{aligned} \quad (9)$$

where $\tilde{\sigma}_t(S)$ is the expected influence under the condition that all $G_t[C_j]$ are strongly connected. Therefore, we have $\tilde{\sigma}_t(S) \leq \left(\prod_{j=1}^\ell \text{Scr}(G_t[C_j])^{-1} \right) \cdot \sigma_t(S)$. $\square$

APPENDIX G FULL EXPERIMENT RESULTS

This section shows the complete experiment results that are omitted in Section IV due to space constraints.

Effect of k . We investigate the impact of the number of seed nodes k on influence persistence, as shown in Figures 13 and 17. Across all networks and parameter settings, RevG, LRep, RevG+, and LRep+ consistently achieve the highest influence persistence, validating the effectiveness of our proposed algorithms. Specifically, compared to the optimal baseline INF, our algorithms achieve up to 100% improvement in total influence persistence. Moreover, influence persistence increases with larger k , as more seeds influence more nodes per snapshot, raising the number of snapshots satisfying threshold α .

Furthermore, we analyze the effect of k on running time, with results shown in Figures 18 and 22. It can be seen that RevG+ and LRep+ consistently outperform RevG and LRep, benefiting from SCC-DAG-based sampling on compressed graphs, which shortens sampling paths and reduces node visits per SCC-RR set, leading to up to a 400% improvement in

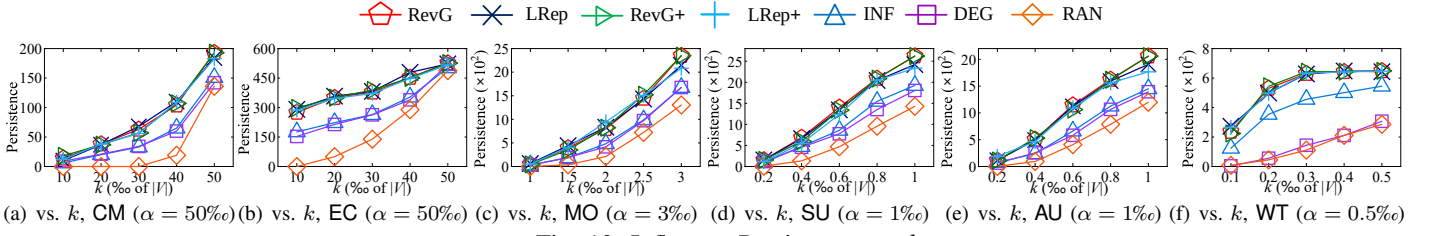
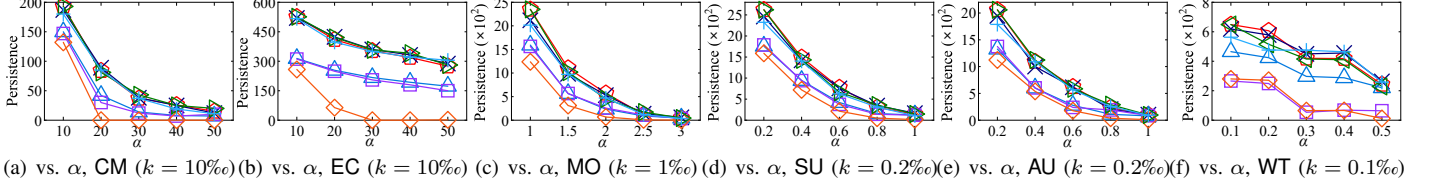
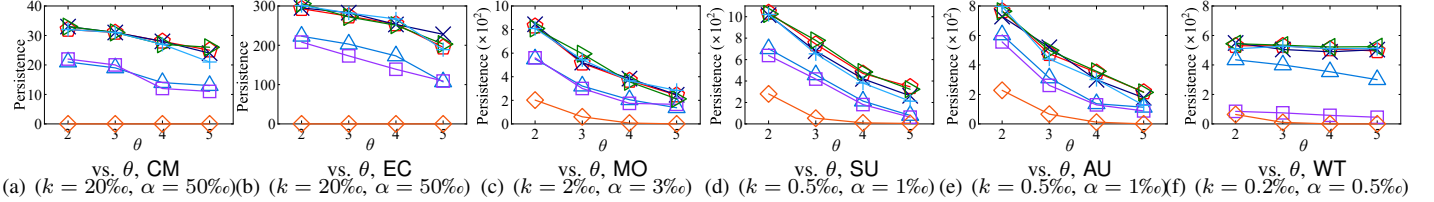
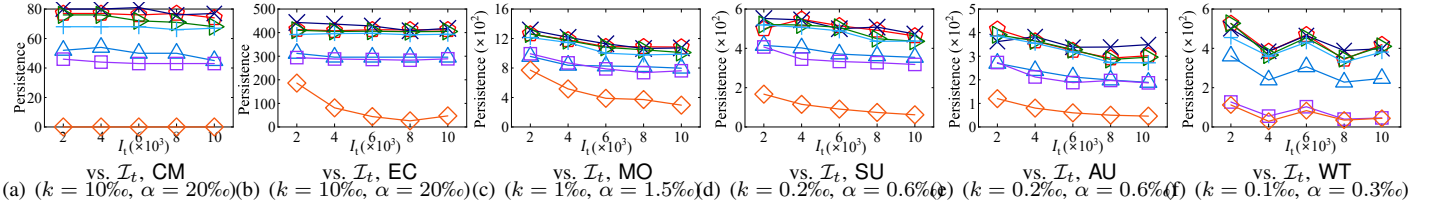
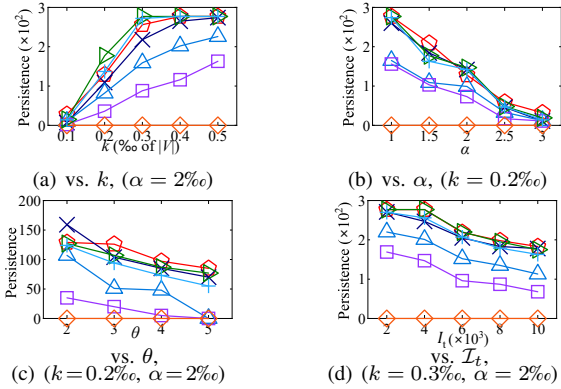
Fig. 13: Influence Persistence vs. k Fig. 14: Influence Persistence vs. α Fig. 15: Influence Persistence vs. θ Fig. 16: Influence Persistence vs. I_t 

Fig. 17: Influence Persistence on SO

efficiency over RevG and LRep. Additionally, as k increases, RevG and RevG+ generally run faster, since the greedy deletion process involves fewer iterations. In contrast, LRep and LRep+ exhibit increased running time with larger k , due to larger initial seed set and more replacements in each iteration. Furthermore, LRep (resp. LRep+) is generally faster than RevG (resp. RevG+) in most cases. This is because LRep (resp. LRep+) prioritizes replacing the weakest seed and terminates early if no improvement in influence persistence is observed. In contrast, RevG (resp. RevG+) exhaustively examines all candidates in each iteration to select one for removal, resulting in a higher

computational cost per iteration.

Effect of α . We study the impact of the influence threshold α and report the influence persistence in Figures 14 and 17. We observe that the influence persistence decreases as α increases. This occurs because, for a fix-sized seed set, a larger α results in fewer snapshots meeting the threshold, thereby reducing influence persistence.

We further investigate the impact of α on algorithm performance, with results in Figures 19 and 22. We can observe that the running times of LRep and LRep+ remain relatively stable on EC and AU, as their runtime is dominated by seed replacement, which primarily depends on the seed budget k . Varying α only affects the computation of persistent intervals, which incurs relatively minor overhead. However, on the larger graph SO, the running times of LRep and LRep+ increase as α grows. This is because higher α values lead to fewer qualifying snapshots, which become more sparsely and discontinuously distributed. This increases the randomness in whether replacing the weakest node improves the influence persistence, thereby leading to more replacements and persistence evaluations. In contrast, the running times of RevG and RevG+ increase with α , because when the candidate set exceeds $\alpha \cdot |V|$, we retain $\alpha \cdot |V|$ candidates. Thus, a larger α results in more candidates being preserved and processed, increasing the total running time.

Effect of θ . We explore the impact of the persistence threshold

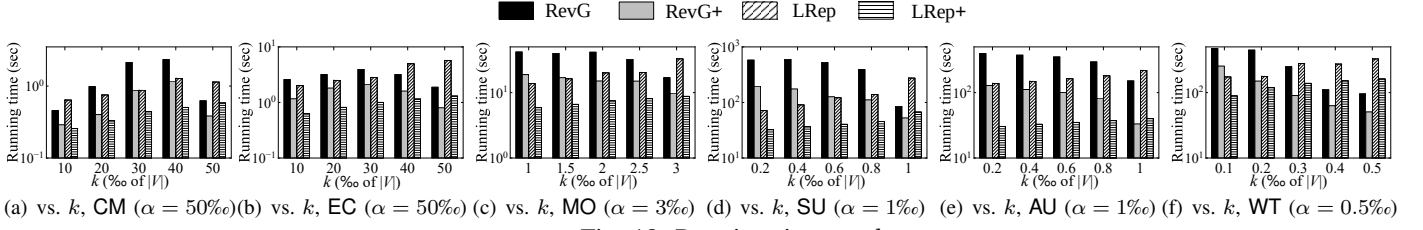
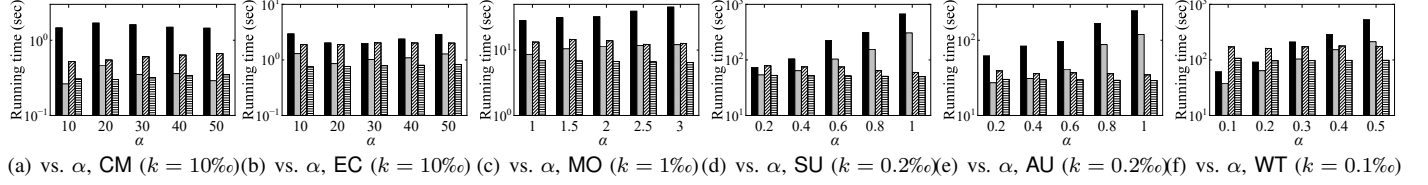
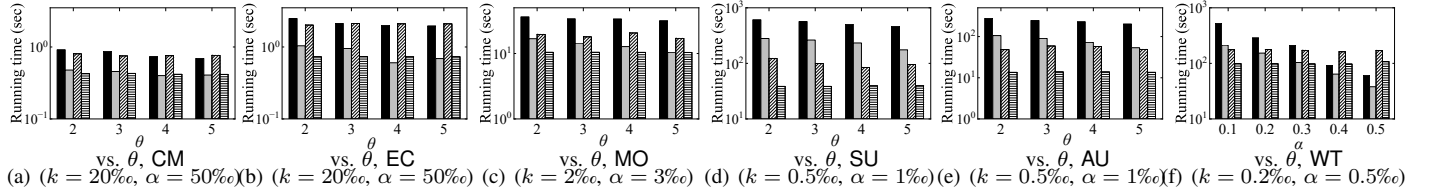
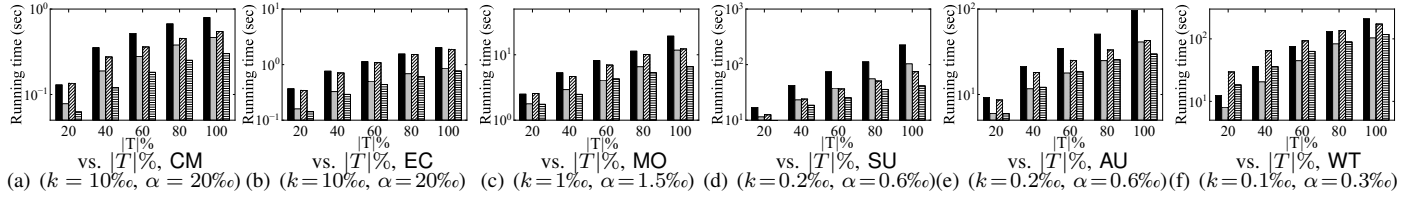
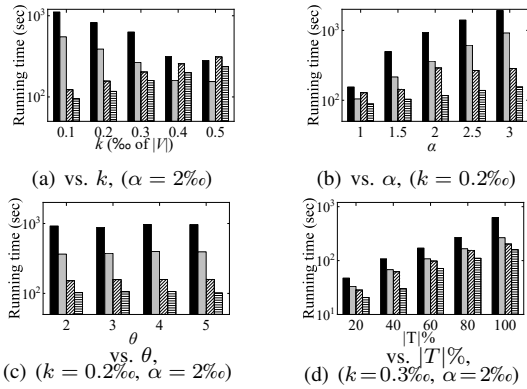
Fig. 18: Running time vs. k Fig. 19: Running time vs. α Fig. 20: Running time vs. θ Fig. 21: Running time vs. T 

Fig. 22: Running time on SO

θ on influence persistence. As shown in Figures 15 and 17, influence persistence decreases as θ grows. This is because, given a fixed set of snapshots meeting threshold α , a higher θ enforces stricter continuity, yielding fewer qualified time intervals and lower overall influence persistence.

Additionally, we evaluate the effect of θ on algorithm efficiency. The results are presented in Figures 20 and 22. We observe that the running times of LRep and LRep+ remain nearly constant, while those of RevG and RevG+ slightly decrease as θ increases. This occurs because larger θ values cause more snapshots to be pruned during candidate set generation, shrinking the candidate set and reducing the time needed for greedy node deletion, thus lowering total runtime.

Effect of $\mathcal{I}_t$. We evaluate the impact of the number of samples per snapshot $\mathcal{I}_t$. The results are presented in Figures 16 and 17. It is observed that as $\mathcal{I}_t$ increases, the influence persistence of all algorithms initially decreases and then stabilizes. This is because a larger $\mathcal{I}_t$ enables a more accurate unbiased estimation of influence spread within each snapshot, thereby yielding more consistent influence persistence.

Effect of T . We study the scalability of our algorithms by varying the time span T , i.e., the number of snapshots. To this end, we retain different fractions of snapshots from the entire temporal graph to generate temporal graphs with different T s. The results are reported in Figure 21 and 22. As expected, both the influence persistence and running time increase when the time span grows. This is because more snapshots lead to more qualified persistent intervals and longer processing time for computing the influence spread across all snapshots.